



Review article

Energy efficient task scheduling based on deep reinforcement learning in cloud environment: A specialized review

Huanhuan Hou^{a,c}, Siti Nuraishah Agos Jawaddi^a, Azlan Ismail^{a,b,*}

^a School of Computing Sciences, College of Computing, Informatics and Mathematics, Universiti Teknologi MARA (UiTM), Shah Alam, 40450, Selangor, Malaysia

^b Institute for Big Data Analytics and Artificial Intelligence (IBDAAI), Universiti Teknologi MARA (UiTM), Shah Alam, 40450, Selangor, Malaysia

^c Department of Computer Engineering, Taiyuan Institute of Technology, Taiyuan, 030008, Shanxi, China

ARTICLE INFO

Keywords:

Task scheduling
Deep reinforcement learning
Energy efficient
Markov decision process
Cloud computing

ABSTRACT

The expanding scale of cloud data centers and the diversification of user services have led to an increase in energy consumption and greenhouse gas emissions, resulting in long-term detrimental effects on the environment. To address this issue, scheduling techniques that reduce energy usage have become a hot topic in cloud computing and cluster management. The Deep Reinforcement Learning (DRL) approach, which combines the advantages of Deep Learning and Reinforcement Learning, has shown promise in resolving scheduling problems in cloud computing. However, reviews of the literature on task scheduling that employ DRL techniques for reducing energy consumption are limited. In this paper, we survey and analyze energy consumption models used for scheduling goals, provide an overview of the DRL algorithms used in the literature, and quantitatively compare the model differences of Markov Decision Process elements. We also summarize the experimental platforms, datasets, and neural network structures used in the DRL algorithm. Finally, we analyze the research gap in DRL-based task scheduling and discuss existing challenges as well as future directions from various aspects. This paper contributes to the correlation perspective on the task scheduling problem with the DRL approach and provides a reference for in-depth research on the direction of DRL-based task scheduling research. Our findings suggest that DRL-based scheduling techniques can significantly reduce energy consumption in cloud data centers, making them a promising area for further investigation.

1. Introduction

Cloud computing has emerged as a reliable and scalable computing model that caters to the needs of individuals and businesses. Cloud service providers receive user requests in the form of applications that can either comprise independent tasks or non-independent tasks such as workflows [1]. Task scheduling refers to assigning tasks with appropriate computing resources to execute based on the Service Level Agreement (SLA). Efficient task scheduling strategies are crucial to satisfy the Quality of Service (QoS) requirement while optimizing various aspects such as time, cost, energy consumption, reliability, etc. [2].

Although cloud computing has become a revolutionary computing paradigm, the significant energy costs associated with the ever-increasing user demands and expansion of the cloud computing data centers pose serious challenges [3]. The high energy consumption translates to elevated costs and increased carbon emissions, both of which are detrimental to the environment and sustainable development [4].

Thus, minimizing the data center electricity consumption for large-scale cloud service providers is imperative.

Previous literature provides comprehensive surveys on task scheduling research based on scheduling methods [5,6] and scheduling objectives [7,8]. The scheduling method can be referred to as heuristic, meta-heuristic, hybrid, or based on Deep Learning (DL) and Reinforcement Learning (RL) [9,10]. Conversely, the scheduling objectives can be associated with the set of metrics such as makespan, execution cost, response time, resource utilization, energy consumption, throughput, reliability, availability, and load balancing [7,8]. These metrics serve as important benchmarks for evaluating and optimizing task scheduling algorithms in different contexts.

With the advancement of artificial intelligence in recent years [11, 12], some scheduling strategies based on Deep Reinforcement Learning (DRL), which combines the perception of DL and the decision-making ability of RL have emerged. DRL-based scheduling methods can automatically learn optimized scheduling policies without prior knowledge,

* Corresponding author at: School of Computing Sciences, College of Computing, Informatics and Mathematics, Universiti Teknologi MARA (UiTM), Shah Alam, 40450, Selangor, Malaysia.

E-mail addresses: huanhuanhou@xmphdss.cn (H. Hou), aishahagos96@gmail.com (S.N. Agos Jawaddi), azlanismail@uitm.edu.my (A. Ismail).

<https://doi.org/10.1016/j.future.2023.10.002>

Received 6 March 2023; Received in revised form 16 August 2023; Accepted 6 October 2023

Available online 14 October 2023

0167-739X/© 2023 Elsevier B.V. All rights reserved.

making them powerful tools in complex cloud computing scenarios. DRL has demonstrated superiorities over some traditional scheduling methods and has shown promising results in addressing the resource scheduling problem in cloud computing [13–19].

Different optimization solutions can be taken at multiple levels to improve data center energy efficiency, such as location-based, infrastructure-based, hardware-based and software-based optimization [20]. Task scheduling is a software design-oriented optimization approach, specifically, belonging to resource management at the software optimization level. There also has been some investigation into methodologies to reduce energy consumption in data centers [21–23]. However, the classifications of energy-efficiency approaches in these studies span a broad range of methods.

The existing surveys have discussed the resource scheduling methods and energy efficiency approaches in cloud data centers. Some researchers also provide valuable reviews of RL/DRL-based resource management of cloud computing. However, few studies have deeply investigated energy efficient task scheduling systematically based on the DRL technique. We complement the existing reviews to address the gap by presenting a detailed analysis and comparison of the various DRL applications in task scheduling problems. Our work is particularly motivated by the following facts:

- The advantages and potential of DRL in solving complex decision-making problems.
- The fact that energy consumption is a pressing concern in cloud data centers.
- The requirement for capturing the quantitative differences between DRL models in the relative literature.

This paper not only analyzes and classifies the energy consumption models used in previous studies but also provides a detailed comparison of DRL algorithms, the experimental environment, workload dataset, and deep neural network structures. This leads us to look into the relationship between scheduling objectives and reward function, which is significant for employing DRL to solve task scheduling issues. Our work attempts to identify the characteristics of various DRL-based methods in the context of energy-aware scheduling and highlight potential research gaps and opportunities for further advancement by a thorough analysis of the literature.

To be specific, the contributions of this review are:

- Quantitative comparison and classification of energy models in the data center of cloud computing.
- Comparison of the components of different DRL methods in terms of algorithm type, scheduling objectives, state space, action space, reward function as well as benchmarks.
- Analysis and suggestion of broad guidelines for reward functions by scheduling objectives, and the proposal of a general framework for a DRL-based task scheduler.
- Comparison and summary of the experimental environment, dataset, and deep neural network structure for task scheduling problems with DRL methods.
- Discussion of gaps, open issues and future research challenges in energy efficient task scheduling with DRL approach.

The remainder of this paper is organized as follows. Section 2 discusses the related surveys on RL/DRL-based scheduling and relative research of energy efficiency in the cloud computing environment. Section 3 describes the research strategy. Section 4 explains the energy consumption models implemented in the existing works. Section 5 provides the review of the energy-efficient task scheduling research with RL/DRL. Section 6 compares the implementation framework of the DRL algorithm, workload dataset as well as the structure of the implementation deep neural network in the previous study. Section 7 presents the gaps, open challenges and suggests future directions. Section 8 concludes this review.

2. Related surveys

2.1. Surveys on RL/DRL-based resource scheduling in cloud computing

There are existing reviews on the topic of RL [24–26], as well as DRL [12] in addressing cloud scheduling problems. Zhou et al. [12] proposed a classification of major approaches for scheduling, and focused on the application of DRL in the scheduling of cloud computing. They compared the research with non-DRL methods and provided the evolution of RL and DRL frameworks in resource scheduling. Shyalika et al. [24] summarized previous works on dynamic task scheduling using reinforcement learning techniques, while Joloudar et al. [25] presented Machine Learning(ML)-based approaches on resource allocation in various computing paradigms: cloud computing, fog computing, edge computing, Internet of Things, Wireless network. Gari et al. [26] gave a comparative summary of the relevant works in the literature that apply RL techniques for scaling and scheduling. They surveyed the scaling and scheduling subproblems based on RL jointly for three types of applications: workflows, independent tasks, and cloud services.

Although these surveys have reviewed scheduling applications based on RL/DRL technology. The absence of an in-depth comparison and analysis of the Markov Decision Process (MDP) model elements in RL/DRL for scheduling applications suggests a gap in the existing literature. Additionally, there seems to be a lack of discussion regarding the experimental datasets, implementation platforms, and neural network architectures utilized to implement DRL-based task scheduling algorithms.

2.2. Surveys on energy efficient techniques in cloud computing

A considerable survey has been made on the energy optimization of cloud computing. Bharany et al. [27] investigated the environmental impact of cloud data centers' excessive energy consumption. The taxonomy of different energy-saving approaches used in data centers has also been examined. Ghafari et al. [22] provided a comparative analysis of 67 scheduling methods in the cloud system to minimize energy consumption during task scheduling. The advantages and limitations of the proposed algorithms are also described to choose the right scheduling algorithm that optimizes energy properly. To lower the amount of electricity required by data centers, Katal et al. [28] also explored several methods utilized at the software level, such as operating systems, virtualization, and applications. But these categorizations of energy-efficiency approaches in their studies cover a wide range of techniques that fail to narrow down the DRL-related applications.

Zhou et al. [29] presented a unified simulation-based analysis framework based on CloudSim that allows the evaluation of energy-efficient VM consolidation algorithms. Five state-of-the-art VM consolidation-based energy-efficient algorithms were compared from multiple perspectives, including core ideas, architectures, modeling, and algorithm complexity. Their research concentrates on analyzing the energy effectiveness of various VM consolidation strategies. In contrast, our work focuses on improving energy efficiency by DRL-based task scheduling in the scope of resource management.

Ismael [30] provided an overview of the literature on energy-aware cloud simulation, including methods for cloud simulation, metrics for measuring energy use, and methods for power management. The energy-consuming equipment, energy consumption measurements, and power management capabilities of the chosen cloud simulators are thoroughly compared in the paper. However, this study does not emphasize comparing DRL-based scheduling simulations for energy efficiency.

Our research focuses on the literature that combines DRL-based scheduling methods with energy efficient task scheduling. We aim to analyze and compare existing approaches in this specific area to gain insights into the effectiveness of DRL algorithms in achieving energy efficient task scheduling.

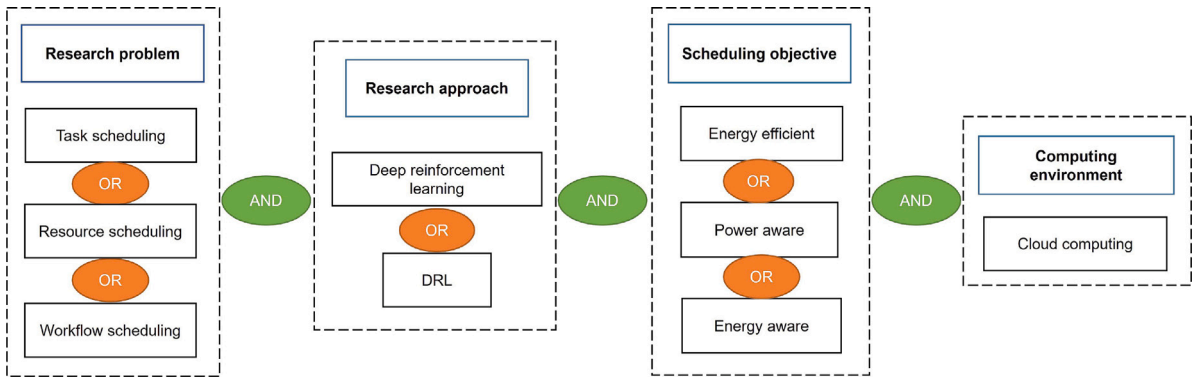


Fig. 1. Search strategy to get relative papers.

3. Research strategy

In this section, we described a systematic literature review that has been referred to study the existing efforts on the research topic, namely, the energy-efficient task scheduling based on DRL. Publications from several electronic database resources have been selected from the years 2018 to 2022 and the resources are as follows:

- Springer (www.springerlink.com)
- ScienceDirect (www.sciencedirect.com)
- IEEE Explore (www.ieeexplore.ieee.org)
- ACM Digital Library (www.acm.org/dl)
- Scopus (www.scopus.com)
- Wiley Library (www.onlinelibrary.wiley.com)

We used combinations of the following terms: “task scheduling”, “resource scheduling”, “workflow scheduling”, “deep reinforcement learning”, “DRL”, “energy efficient”, “power-aware”, “energy aware”, and “cloud computing”. Logical operators like AND, and OR were used in our search to connect the keywords for accessing more research papers. Fig. 1 shows the search strategy we used.

The initial search yielded 255 papers, including 58 from Scopus, 69 from ScienceDirect, 33 from ACM, 47 from Springer, 38 from IEEE, and 10 from Wiley. The distribution of the searched papers from different publications is illustrated in Fig. 2. After a careful manual review of the research papers based on keywords and exclusion criteria, we finally identified 16 core research papers for comparison with the DRL methodology in our survey. The exclusion criteria were: non-English papers with few citations were excluded from this review, and research focus on “VM consolidation”, “VM migration” and “task offloading” was also out of the scope of our search. For the breakdown of energy consumption models, we specifically chose a representative selection of 20 papers, some of which also appear in the DRL comparative literature.

By investigating the literature on task scheduling for energy reduction using the DRL approach, we aim to figure out the following research questions. Table 1 describes the list of research questions required to investigate through a review of the literature.

4. Energy consumption model

Establishing an accurate energy consumption model is a prerequisite for obtaining an optimization algorithm. Energy consumption in a data center is generated in multiple ways, including IT equipment, power supply systems, and cooling systems [31]. Measuring the energy usage of each partition in a practical setting is challenging, but power consumption is a more quantifiable metric.

For task scheduling, the energy model usually considers only the power consumption of computing resources during the execution of tasks. Numerous studies reveal a linear relationship between CPU

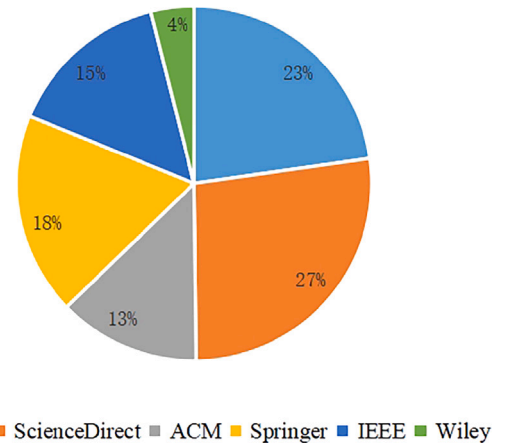


Fig. 2. Distribution of papers from different publications.

Table 1
Research questions (RQ).

NO.	Questions
1	Which energy consumption models are used in task scheduling studies and what is the difference between them?
2	What DRL algorithms are used by researchers in energy efficient task scheduling problems?
3	How to define elements of the MDP model: state space, action space, and reward function in the literature?
4	What is the relationship between reward function and energy consumption model?
5	What experiment platform or deep learning frameworks are used to implement DRL-based scheduling algorithms?
6	What datasets are used as the workload for task scheduling?
7	What is the neural network structure of the agent in DRL based scheduling algorithm?

load and physical server power consumption, despite the fact, such as memory, disk, and I/O that other factors also play a role [32,33].

We address RQ1 by investigating the energy consumption models used in the literature and classifying energy consumption models into two main approaches: the first approach is based on combining the static and dynamic power of the machines or servers in the cloud data center, whereas the other depends on the power consumption statistics that SPECpower has released for various physical computers working at varying CPU loads. The classification of the energy consumption calculation approach is shown in Fig. 3. In this section, we provide the answer to RQ1.

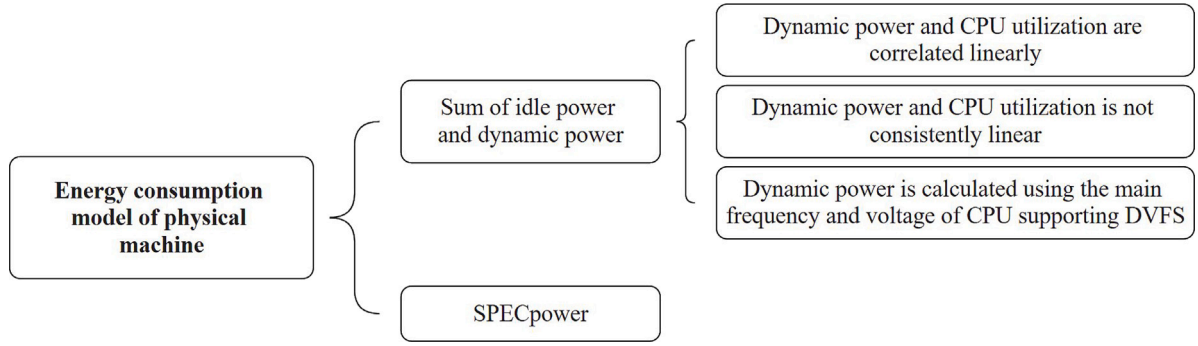


Fig. 3. Classification of energy consumption model.

Table 2
Variables related to energy consumption.

Notation	Description
H_i	Host i
$P_{dynamic_i}(t)$	Dynamic power consumption of host i at time t
$U_{c_i}(t)$	CPU utilization rate of host i at current time t
$U_{c_{opt}_i}$	Optimal CPU utilization rate of host i
$U_{c_{idle}_i}$	CPU utilization rate of host i at idle state
$U_{c_{max}_i}$	CPU utilization rate of host i at full load state
P_{idle}_i	Power consumption of host i at idle state
P_{max}_i	Power consumption of host i at full load state
$P_i(t)$	Power consumption of host i at time t
$E(i)$	Total power consumption of host i in time period
E	Total power consumption of all host in data center

4.1. Combination of idle power and dynamic power of physical machine

In the first approach, the calculation of energy consumption is based on the physical machine. Each physical machine has two states: idle and active. Idle means that the physical machine does not perform any tasks and only retains power for the static operation of the system, such as keeping the clock running and maintaining the basic circuits, and the idle power of the physical machine H_i is a constant, denoted as P_{idle}_i . Active means that tasks are executed on the physical machine and the power of the active state is denoted by $P_{dynamic_i}(t)$. The energy consumption value at a certain moment on a physical machine is obtained by adding idle and dynamic components.

In addition, the CPU utilization in the idle state is denoted as $U_{c_{idle}_i}$, the maximum CPU load (utilization at full load) is denoted as $U_{c_{max}_i}$, and the current CPU utilization can be calculated by dividing the allocated CPUs on the current physical machine by the total CPU capacity, denoted as $U_{c_i}(t)$. Table 2 shows the symbols of variables related to energy consumption.

The differences between the power consumption models in the research literature are mainly due to the different calculations of dynamic power. Table 3 summarizes the formula of dynamic power consumption models used in the related works.

1. The dynamic power consumption and the current CPU utilization are correlated linearly. However, there is a slight difference in the form of parameters. One form is that both the coefficient and bias in a linear relationship are constants [16,34], as shown in Eq. (1). Another way of expressing is adopting the power consumption of the host at full load minus the power consumption at idle as the coefficient of the linear relationship, without taking the bias, as indicated by Eq. (2). Existing works, namely, [35–40], applied this approach to estimate the dynamic power consumption of a server.
2. The relationship between dynamic power consumption and CPU utilization is not consistently linear. Dynamic power is linearly increased when the current CPU utilization rate is under the

optimal utilization and non-linearly increased otherwise. In the research of [3,41–43], Eq. (3) illustrates how this type of model is utilized to evaluate the dynamic power.

3. Dynamic power consumption is calculated using the main frequency and voltage of the CPU supporting the Dynamic Voltage and Frequency Scaling (DVFS) technique. DVFS has been widely used in CMOS-based processors for energy saving and hardware heat production control by scaling CPU voltage and frequency in real-time in a highly scalable way [47]. Usually, a CPU can support multiple running frequencies. CMOS-based processor power consumption per time unit is highly related to support voltage and frequency. [32,33,44–46] used the same model to obtain dynamic power value. Eq. (4) has been used in the literature to represent the dynamic power consumption of a physical machine based on DVFS technology.

For the first and second cases of calculating dynamic power, according to Eq. (5), we can get the power consumption of the host at time t .

$$P_i(t) = P_{idle}_i + P_{dynamic_i}(t) \quad (5)$$

It is worth noting that, compared to the previous two models, one more item is added to the calculation of physical power consumption in the third case: the host independent energy consumption denoted as P_{ind}_i . This component is free from CPU frequency and supply voltage and it can be minimized by keeping the system in sleep mode [33]. In this case, the formula for calculating dynamic power is represented as Eq. (6).

$$P_i(t) = P_{idle}_i + h(P_{dynamic_{i(t)}} + P_{ind}_i) \quad (6)$$

where h describes the state of the system, which is 0 to specify the system is in sleeping mode and 1 for active mode.

The power consumption obtained by Eq. (5) or Eq. (6) is only the value at a certain moment. The workload of a physical machine is a continuous value dynamically changing from time to time, resulting in a fluctuation of CPU utilization rate. Therefore the energy consumption of host H_i in the period time $[t1, t2]$ can be denoted as an integral function of $P_i(t)$, which is formulated as Eq. (7)

$$E(i) = \int_{t1}^{t2} P_i(t) dt \quad (7)$$

Finally, the total energy consumption of all hosts can be formulated as in Eq. (8), as follows:

$$E = \sum_{i=1}^N E(i) \quad (8)$$

4.2. SPECpower

Another type of approach, instead of using an analytical model of power consumption by a physical machine, researchers use real data on

Table 3
Formula of dynamic power models.

Author	Method	Dynamic Power Formula	
[16,34]	1	$P_{dynamic_i}(t) = \beta * U_{c_{li}} + \alpha$	(1)
[35–40]	1	$P_{dynamic_i}(t) = (P_{i_{max}} - P_{i_{idle}}) * U_{c_{li}}$	(2)
[3,41–43]	2	$P_{dynamic_i}(t) = \begin{cases} \beta * U_{c_{li}}, & U_{c_{li}} < U_{c_{opt_i}} \\ \beta * U_{c_{opt_i}} + \alpha * (U_{c_{li}} - U_{c_{opt_i}})^2, & U_{c_{li}} \geq U_{c_{opt_i}} \end{cases}$	(3)
[32,33,44–46]	3	$P_{dynamic_i}(t) = C_{eff} V^2 f$ C_{eff} is the effective-load capacitance, f and V are frequency and voltage respectively	(4)

Table 4
Energy consumption at different load levels in Watts.

Vendor	System	Idle	10%	20%	30%	40%	50%	60%	70%	80%	90%	100%
Fujitsu	TX1330 M2	16.1	19.3	20.6	22.3	24.2	27	30.5	34.7	39.2	43.9	47.2
Huawei	RH2288H V2	68.7	78.3	84	88.4	92.5	97.3	104	111	121	131	137
Acer	Acer AR580 F2	134	172	202	239	286	346	405	452	490	582	667
Acer	Acer AR580 F2	134	172	202	239	286	346	405	452	490	582	667
Acer	Acer AR180 F1	76.4	125	137	148	163	180	199	217	233	248	265
Acer	Acer AR360 F2r	69.4	103	117	137	159	182	201	218	240	290	315
Dell Inc.	PowerEdge R720	43.9	67.4	76.5	85.4	94.5	103	114	124	134	148	173
Sugon	I620-G30	51.9	107	117	127	137	146	157	172	200	227	245
Lenovo	ThinkSystem SR63	554	99.6	112	123	132	143	154	165	178	203	240
Sugon	Sugon I620-G20	52.5	91.3	107	122	139	156	175	196	217	240	264
IBM	X3755 M3	122	200	244	294	345	393	446	475	514	550	580

power consumption provided by the results of the SPECpower benchmark. The energy consumption model provided by SPECpower for various servers includes energy consumption during idle time as well as energy consumption with different CPU load values ranging from 10% to 100%. Table 4 displays the energy usage of ten distinct types of heterogeneous servers at various load levels, evaluated using actual data from the SPECpower test results [48]. Based on the SPECpower energy model, it is usually necessary to process the discrete CPU load and energy consumption values, for example, by interpolation or linear regression to obtain a continuous energy consumption function, because in real cloud computing environments, the CPU load of a host can be any value between 0 and 100 percent.

The work by [49] selected three recent models with CPU from Intel and AMD measurement results published according to SPECpower. They designed a segmented linear relationship according to CPU utilization in each 10% interval to improve the accuracy of the energy consumption model. In [48], they utilized regression models to fit the energy consumption model with the least loss based on the SPECpower benchmark. By reducing the squared loss value, the nonlinear operators are identified and a fixed function model is fitted which can estimate power consumption at various load intensities. The host power consumption models of different manufacturers can be retrieved on the SPEC website [50].

After obtaining the energy consumption value at any CPU load by using a fitting method based on the SPECpower benchmark, as in the first approach, Eq. (7) can be used to calculate the energy consumption for a host at a specific time zone. Eq. (8) can then be used to get the total energy consumption values for all hosts in the data center.

5. Review of the energy efficient task scheduling research with RL/DRL

In addition to exclusion rules mentioned in Section 3, we have narrowed down 16 core papers from the research articles based on the following criteria: (1) To make the comparison more persuasive, the evaluated algorithms were published in prominent journals or conferences, and they are picked from state-of-the-art task scheduling algorithms with DRL techniques that have shown good performance in energy efficiency. (2) To ensure the comparability of the key characteristics of MDP elements, the corresponding parts of the literature need to be expressed clearly and completely.

This section provides a comparison to help readers fully comprehend the current DRL-based task scheduling for energy efficiency. The RL/DRL algorithms are briefly described and categorized in Section 5.1. Followed by a comprehensive comparison from several perspectives: algorithm, task type, scheduling objective, elements of MDP, as well as the baseline used in the studies in Section 5.2, which also provides the answers to RQ2 and RQ3. A discussion of the relationship between the scheduling objective and the rewards function is provided in Section 5.3, which addresses RQ4. A general DRL-based task scheduling framework is proposed in Section 5.4.

5.1. Overview of RL and DRL algorithms

RL is a computational approach that facilitates goal-directed learning and decision-making problems [51]. It provides a framework to learn from interactions to achieve a specific objective. A general framework of RL is illustrated in Fig. 4 [51]. The agent acts as the decision-maker and learner, while the environment comprises everything outside the agent with which the agent interacts. Interactions between the agent and environment occur at each discrete time step in a sequence. At time step t , the agent is given a description of the environment's state S_t , chooses an action A_t based on that information, and receives a reward R_t one-time step later, as a result of that action. The environment then presents the agent with a new state S_{t+1} . The agent's objective is to maximize its overall reward over the long term. [52] adopted the RL approach to dynamically switch off microservices and schedule battery usage, which first explored the optimization of renewable energy use in cloud data centers.

MDP is commonly utilized for modeling the environment in which the RL agent operates. The RL algorithms can be categorized into two types, model-based and model-free, depending on whether the elements of the MDP model are known.

The optimal policy in the case of a model-based can be obtained by iteratively calculating the optimal value function based on the Bellman equation, such as the Dynamic Programming approach. Although Dynamic Programming methods are well-developed mathematically, they require a complete and accurate model of the environment.

In many practical applications, models are not available, and the environment requires simulation to explore. One technique that can be used is the Monte Carlo method, which computes numerical characteristics or parameters through observation or sampling tests of the

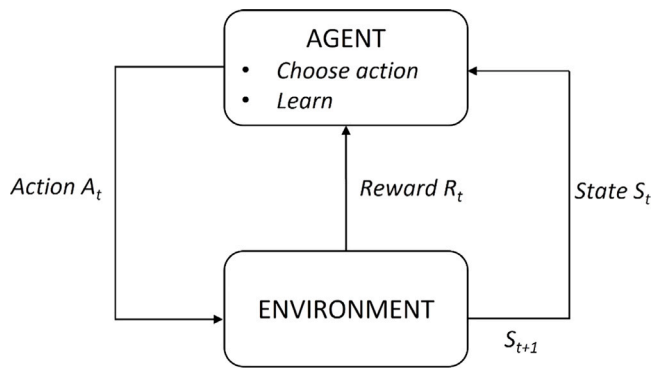


Fig. 4. A general framework of reinforcement learning.

model or process and provides an approximation of the solution. Monte Carlo methods do not necessitate a model and are conceptually straightforward, but they may not be well-suited for step-by-step incremental computation.

Another method that can be used is function approximation. Temporal-Difference (TD) learning is a combination of Monte Carlo ideas and Dynamic Programming ideas [51]. Without the model of the dynamic environment, TD approaches can learn directly from raw experience, and they can update estimates based on previously learned results without waiting for the end of the episode, as in Monte Carlo. Q-learning and SARSA are the most fundamental TD algorithms in reinforcement learning. However, these techniques require the agent to visit all states during the training process and store the state transition data in a tabular format. They may be insufficient for handling complex environments with large dimensional or continuous state–action spaces.

DRL has emerged as a promising technique that combines the benefits of DL and RL to address the problem of energy consumption in cloud data centers. By employing function approximation, DRL leverages Deep Neural Networks (DNNs) to approximate the parameters of the optimal value function or optimal policy function. The two main branches of Model-free in RL are Value-based and Policy-based. The former approximates the action value function with neural networks and has been successfully applied to master difficult control policies in high-dimensional state–action space by using Deep Q Learning (DQN) [53]. Several enhanced variants of DQN have emerged, including DQN with Prioritized Experience Replay (DQN+PER) [54], Double DQN [55], and Dueling DQN [56], which have shown better performance compared to the original DQN.

In contrast, Policy-based is an alternative learning approach that directly learns a parameterized policy for selecting actions without consulting a value function. REINFORCE is a well-known Monte Carlo Policy-Gradient Method but is only well-defined for the episodic case, with all updates made retrospectively after the episode is completed. However, the delayed convergence process makes this basic strategy impractical due to high variance. To address this issue, introducing a baseline is a typical method of reducing variance.

The Actor–Critic(AC) approach is a method that combines both learning policy and value function approximations. It consists of two components: the Actor, which learns the policy, and the Critic, which learns the value function. Advantage Actor–Critic(A2C) is an Actor–Critic method that uses the state value function as a baseline to reduce variance. Asynchronous Advantage Actor–Critic(A3C) [57] is an extension of A2C that uses multiple agents to asynchronously execute in parallel, each with its parameters and a copy of the environment. Proximal Policy Optimization(PPO) is an off-policy version of Policy Gradient based on the Actor–Critic framework. It is derived from Trust Region Policy Optimization(TRPO) [58], which limits the amount of variation in the policy gradient so that the loss function is monotonically increasing. Google and OpenAI proposed their PPO algorithms

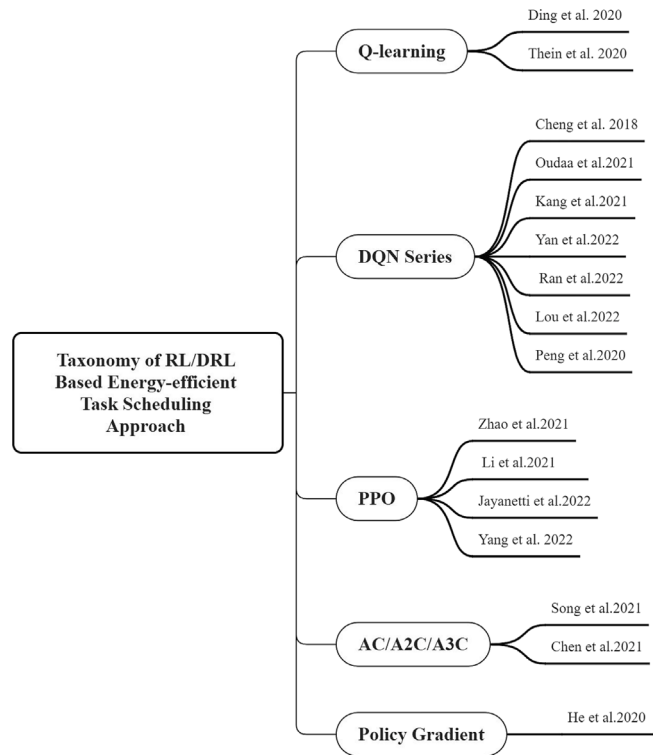


Fig. 5. Taxonomy of RL/DRL based energy efficient task scheduling algorithms.

respectively: Distributed version of PPO(DDPPO) [59] and PPO [60]. Additionally, Deep Deterministic Policy Gradient(DDPG) [61] has been proposed to address the problem of continuous action control. The Actor–Critic structure used by DDPG, in combination with the successful DQN architecture, has enhanced the stability and convergence of the Actor–Critic approach.

5.2. Comparison of DRL algorithms applied to task scheduling

The above provides an overview of the main types of algorithms of DRL. Understanding how these algorithms are used to solve the energy efficient task scheduling problem is more significant. This section not only presents the different DRL algorithms used in previous studies but also provides an in-depth comparison of the definition of the MDP model elements. Table 5 depicts the algorithm, task model, scheduling objective, elements of MDP as well as the benchmark used for comparisons and experimental results in relative literature. We summarize and analyze some points according to the contents of Table 5.

1. Regarding the DRL algorithms

Fig. 5 illustrates the taxonomy of RL/DRL-based energy efficient task scheduling algorithms employed in the literature. DQN and associated enhanced algorithms based on DQN were employed in most papers of these studies (7 articles), followed by the PPO algorithm (4 articles), AC/A2C/A3C algorithm and Q-learning algorithm (2 articles respectively), and Policy gradient (1 article). Various characteristics of task scheduling applications are briefly summarized below.

- **Q-Learning**: Ding et al. [16] proposed a task scheduling framework called QEEC which has two phases for energy-efficient cloud computing. In the second phase, a Q-learning-based scheduler was used to assign tasks to virtual machines, applying incentives to reward the assignments that can reduce task response time and increase CPU

Table 5

Synthetic comparison of study on energy efficient task scheduling based on RL/DRL.

Author	DRL algorithm	Task model	Scheduling objectives	State space	Action space	Reward function	Benchmark
[48]	PPO-Clip	N/A	Reduce energy consumption and ensure the QoS without significantly increasing the waiting time	Partition-level: the union of partition characteristics, servers and characteristic of the task; Server-level: CPU usage of each server and characteristic of task	Partition-level action: the index of partitions; Server-level action: the index of the servers	$Reward_1 = -w1 * Energy_p - w2 * t_{wait}$ (10) $Reward_2 = -w3 * Energy_s - w4 * t_{wait}$ (11) $Energy_p$ is real system energy consumption; $Energy_s$ is the overall energy consumption; t_{wait} is the waiting time of the task; $w3, w4$ are relative weights.	Tetris, H2O-Cloud
[16]	Q-learning	Independent task	Reduce energy consumption	S is a finite set of states that the server can be in. It includes the number of tasks waiting in the buffer queue of each VM at time t.	Action space A is a set of actions that can be performed. An action can have one and only one element taking the value of 1.	$Reward = \begin{cases} 1 & \text{When } wt + et < deadline, \text{ reward} = 0; \\ 0 & \text{When } wt + et < deadline \& wt = \min\{wt\} \& U^c \text{ of vm to assign is max, reward} = 1; \\ -1 & \text{Otherwise reward} = -1 \end{cases}$ (12)	Random, Fair, Greedy
[3]	DQN	DAG	Energy cost, run-time and reject task number	The combination of current server observation and current task observation	Stage1: the collection of all server farms, Stage2: the collection of all servers	$Reward = price(pu(t)) - price(pu(t-1))$ (13) Current energy cost minus previous energy cost	Greedy, RR, FERPTS
[41]	Quantile Regression DQN	DAG	Minimize energy cost efficiency and low task reject rate	Same with [3]	Same with [3]	Same with [3]	Round-Robin
[49]	PPO	N/A	Maximize renewable energy while satisfying deadline constraints	Continuous state space: a large vector of float numbers normalized to $[-1,1]$. The elements of the vector are: time of day, renewable energy, DC power consumption, incoming job, server load, public cloud cost.	Discrete action space: Action 0: The scheduler does nothing; Action $[1, m+1]$: map the task to the i th server in m th private data center; Action $[m+1, n]$: send the task to a type of public cloud VM	$Reward = a * REU + b * CCR + c * DV + d * MR + e * UR + f * TR(a, b, c, d, e, f \geq 0 \text{ and } a + b + c + d + e + f = 1)$ (14) REU means Renewable Energy Utilization; CCR is Cloud Cost Reward; DV is Deadline Violation; MR is Makespan Reward; UR is Utilization Reward; TR is Task Reward; each individual reward are normalized in $[-1,1]$ space	Round Robin (RR), Maximum Renewable Least Utilization First (MR-LUF), DQN.
[35]	Soft Actor-Critic	N/A	Resource usage, energy consumption, and the waiting time of tasks	A vector consists request task, current available resources, CPU utilization of server.	Collection of index of all servers	$Reward = a - \beta 1 * ok - \beta 2 * \sum_{i=1}^D \ln(1 + \exp(r_{i,j}^{surplus} - limit)) - \beta 3 * \sum_{i=1}^N \sum_{j=1}^D \ln(1 + \exp(r_{i,j}^{surplus} - limit)) - \beta 4 * P$ (15) a is the reward when the tasks are assigned successfully; $limit$ is the upper limit of resources; ok is the waiting time of the task T_i ; $\beta 1$ is the weights of the waiting time penalty; $\beta 2$ is the weights of the penalty of resource overloading of the target servers; $\beta 3$ is the weights of the penalty of the resource utilization; $\beta 4$ is the weights of the penalty of power consumption.	Random, DQN
[62]	DQN	Independent task	Minimize energy consumption with keeping QoS	State space contains all possible states of jobs and executing job queue of each VM at time t.	The collection of IDs of free VMs in the cloud environment.	$Reward = (1 + e^{-cost_j}) * \frac{JQoS_j}{T_j^{res}}$ (16) T_j^{res} is the response time of the current job; $JQoS_j$ is the QoS; $cost_j$ is the energy cost to process this job; a is a hyper-parameter used to trade off the QoS and energy cost	Random, Round-Robin, Earliest
[36]	Parameterized Action Space DQN	N/A	Minimize energy consumption	The current state consists of parameters in four dimensions: the number of required CPU cores, the airflow rates, the IT workload states, the thermal states.	A vector consists of server ID and the airflow rate.	$Reward = r_0 - P - \beta$ (17) r_0 is a large enough constant to ensure the reward remains positive; P is joint optimization object; β denotes a penalty for invalid actions that violate the constraints	DQN
[34]	DQN	N/A	Minimize task response time and maximize resource utilization, reduce energy consumption	S is a set of states includes the current resource usage of each server.	A collection of server ID in cloud system.	$Reward = \omega 1 (P_t - P_t^{min}) + \omega 2 (W_t - W_t^{min})$ (18) P_t is the total power consumption of the whole cloud system; P_t^{min} is the expected minimal power consumption; W_t is the response time of task T_j ; W_t^{min} is the expected minimal response time	DQN RR, Hierarchical DRL, DRL-Cloud and MO-DQN.
[63]	A3C	N/A	Reduce average job latency, job dismissing rate, average energy consumption of jobs	State space consists of the resource usage of all servers and resource requests of all arrived jobs by timestep t.	A vector consist 0 and 1 that indicates whether a job will be processed by a server or not.	$Reward = -R_t^{QoS} - R_t^{energy}$ (19) R_t^{QoS} reflects the penalties for different types of latency at timestep t ; R_t^{energy} reflects the penalty for energy consumption at timestep t	LJF, Tetris, SJF, RR, PG, DQN
[38]	DQN	N/A	Reduce energy consumed while maintaining an acceptable QoS	System state is the union of the server cluster state and task state. State space consists of all possible states.	Collection of index of the target server.	$Reward = \frac{E_{total}(t_{j+1}) - E_{total}(t_j)}{t_{j+1} - t_j} - \frac{E_{total}(t_j)}{t_j - t_{j-1}}$ (20) $E_{total}(t_j)$ is the energy consumption during the epoch between t_j and t_{j+1}	DTA, DTM, BF, MAD, RR
[64]	PPO	Workflow	Reducing the energy consumption of the system while maintaining deadline	State space includes the real time status of the network, resource requirements and deadline of the task to be scheduled.	A vector consists the layer which the task should be allocated and the node should be assigned for task.	$Reward = a * M_n + (1 - a) * E_n$ (21) M_n is makespan during the execution of the workflow n ; (if makespan is before Deadline, $M_n = 1$; Else $M_n = -1$.) E_n is energy consumed during the execution of the workflow n	Random, EFT, EDA, EES, Single-Actor
[40]	Policy Gradient with baseline	Independent tasks and DAG	Minimize energy consumption	A list of server-taskInstance pairs	A set of indexes of all pairs, the size is linear to the length of fit server-taskInstance pair list.	$Reward = -1 * E_t$ (22) E_t is the power consumption at time step t	Random, FIRST FIT, TETRIS.
[65]	PPO	Independent tasks and DAG	Minimize energy consumption	A list of server-taskInstance pairs	A set of indexes of all pairs, the size is linear to the length of fit server-taskInstance pair list.	$Reward = -1 * E_t$ (23) E_t is the power consumption at time step t	Random, FIRST FIT, TETRIS, DeepEnergyJS
[66]	Q-learning	N/A	Energy consumption, resource utilization	State space accounts all hosts in general, the state of the candidate host, has four dimensions: current energy consumption, candidate host, utilization of candidate host, CPU demand of virtual machine.	The possible actions for any candidate host can be either ALLOCATE or NOT-ALLOCATE.	Calculate composite reward value for allocation with Fuzzy Logic	N/A
[42]	DQN	DAG	Reduce energy consumption and task makespan	The collection of available virtual machines on the server and the waiting time required for the task to be deployed on the server	Collection of the index of all server	$Reward = a * Normal(R_{makespan}) + (1 - a) * Normal(R_p)$ (24) $R_{makespan}$ is a reward for minimizing the task makespan; R_p is reward for minimizing energy consumption which equals the total energy consumption at current time t minus the total consumption of previous time $t-1$	Random, RR, Multi-objective PSO

utilization on each server. Thein et al. [66] utilized Q-learning to train an energy-efficient resource allocator to select the optimal decision according to the outcome of the host utilization and data center energy efficiency.

- **DQN series:** DQN is the most fundamental algorithm for value-based learning in deep reinforcement learning. Cheng et al. [3] developed a two-stage RP-TS processor using the DQN algorithm with experience replay technique to automatically make the optimal long-term decisions by learning from the changing environment, such as user request patterns and realistic electric prices. Oudaa et al. [41] applied a Quantile Regression-DQN algorithm to train the system's decision-making agent that minimizes long-term energy consumption costs and reject rate while ensuring QoS. Kang et al. [34] suggested an adaptive DRL-based architecture for energy-aware task scheduling taking into account the workloads' unpredictable and ever-changing nature. Yan et al. [62] proposed a DRL-based job scheduling algorithm to reduce the energy consumption of task execution while maintaining high QoS for real-time incoming jobs in cloud data centers. Ran et al. [36] offered the DeepEE optimization framework, which extends the original Deep Q-Network (DQN) with parameterized action space (PADQN) to make decisions for task scheduling (discrete action) and cooling regulating (continuous action). Lou et al. [38] presented an online DRL-based task scheduling algorithm (ODTS), which jointly takes task assignment and migration into account during the lifespan of tasks to plan mixed duration workloads in data centers in an energy-efficient manner. Peng et al. [42] suggested a DQN-based architecture for online resource scheduling by adjusting the amount of reward for different optimization objectives, compromising between the two optimization objectives of energy consumption and job duration.
- **Policy Gradient:** He et al. [40] extended previous work in DeepJS [67] to propose a more effective strategy to improve energy efficiency in heterogeneous cloud computing. The job scheduling problem is considered a multi-dimensional bin packing problem, and a policy gradient algorithm with a baseline is used to train the agent.
- **PPO:** Zhao et al. [49] proposed a Proximal Policy Optimization (PPO)-based Deep Reinforcement Learning (DRL) strategy for maximizing the use of renewable energy in a hybrid multi-cloud environment. To ensure the stability of the training process and facilitate debugging, they normalized the state space to $[-1,1]$. They also defined and normalized the reward function as floating-point numbers in the $[-1,1]$ space, which has been shown to work well in previous research. Experimental results showed that the proposed PPO-based algorithm outperformed the Deep Q-Network (DQN) algorithm in terms of energy consumption under a cost budget. Li et al. [48] proposed a partition-based task-scheduling algorithm using PPO on heterogeneous resources. The work is based on the PPO-Clip method, which first chooses the best heterogeneous partition depending on the current state and then chooses the best server inside the partition. Jayanetti et al. [64] formulated a unique hierarchical state space and proposed a hybrid Actor-Critic technique for energy-efficient resource scheduling in an edge-cloud context. They used the PPO algorithm to overcome the acknowledged drawbacks of conventional Actor-Critic systems. Experimental results showed that the multi-actor technique outperformed the single-actor in terms of performance. Yang et al. [65] implemented the PPO algorithm to train the agent, resulting in an improved version of DeepEnergyJS [40] called DeepEnergyJSV2.0. The findings showed that PPO outperformed the REINFORCE algorithm during training.

- **AC/A2C/A3C:** In [35], the Wolpertinger Scheduling Schemes (WSS) is introduced to improve the efficiency of decision-making on a large discrete action space and address slow DRL training convergence. The WSS system includes components such as the K-nearest neighbor algorithm, soft Q-network, policy network, and soft value network. The Soft Actor-Critic (SAC) algorithm is employed to enhance the stability and exploration capability of WSS. The study reports that WSS can save close to 25% of energy and has a faster convergence speed than DQN. To address the issues with A2C, a low-variance and high-efficiency A3C algorithm is proposed in [63]. The paper proposes an A3C-based resource allocation method for learning optimal policies for job scheduling in cloud data centers. The approach uses multiple agents that interact with the environment in parallel to fully utilize computing resources and speed up learning.

2. Regarding the task type

In cloud computing, user task requests as workloads can be grouped into two types: independent tasks and dependent tasks [68]. The former is generated randomly from different user requests, without any sequential relationship between tasks. In the latter, there are specific dependencies between tasks, such as workflows, which commonly include hundreds of data-intensive or compute-intensive tasks with varying duration and resource requirements. Generally, tasks with dependencies are represented by a Directed Acyclic Graph (DAG). Accordingly, the scheduling issue is significantly made more challenging by the dependence constraint between tasks. Among these studies, independent tasks are used as workloads in their experiments [16, 62], while [3, 41, 42, 64, 65] use the DAG as their workload to a simulation scheduling algorithm. [65] not only handles independent workload model but also randomly constructs DAG topology for each job to verify their proposed algorithm.

3. Regarding the scheduling objectives

Scheduling objectives are quantitative measures used to assess and evaluate the performance, efficiency, and effectiveness of various scheduling algorithms and strategies. All of the sixteen studies cover the scheduling objective of reducing energy consumption or reducing energy costs. [16, 40], and [65] are for the single objective of reducing energy consumption, while the others contain two or more optimal objectives. In these studies, resource utilization and QoS are the most considered optimization objectives apart from energy consumption, followed by other indicators: waiting time, task reject rate and deadline. Other metrics such as makespan, running time, job latency as well as job dismissing rate are also considered to validate the performance of the algorithm. These data may be interpreted that energy consumption is closely related to resource utilization, while various time-related metrics usually determine the quality of service in cloud computing. Therefore, these objectives are usually considered optimization objectives along with the energy consumption values. Here are the definitions of the metrics used in the literature:

- **QoS:** QoS refers to the set of measures, policies, and mechanisms that ensure the consistent and predictable performance of cloud services and applications. It involves maintaining a high level of service quality, resource allocation, and performance guarantees to meet the requirements and expectations of users and applications.
- **Resource Utilization:** This metric measures how effectively the available resources, such as virtual machines or containers, are being used. Higher resource utilization usually indicates better efficiency in task scheduling.

- **Makespan:** The makespan is the total time taken to complete all tasks in a scheduling scenario. Minimizing makespan is often a key objective of scheduling algorithms, as it indicates efficient resource utilization and quicker task completion.
- **Response Time:** It is the time taken between submitting a task and receiving its output. Shorter response times are desirable, as they provide faster results to users or applications.
- **Running time:** It refers to the amount of time it takes for a specific task, program, or computation to complete its execution on a computing system.
- **Waiting time:** It refers to the amount of time that a task or job spends in a queue waiting for its turn to be executed on available resources.
- **Task rejection rate:** It refers to the percentage of tasks or jobs that are rejected by a scheduling system or framework due to various reasons.
- **Job latency:** It measures the delay experienced by a task from the time it enters the scheduling queue until it finishes execution and produces its output.
- **Deadline:** When tasks are associated with deadlines, it means that they must be executed and completed by the designated time to meet performance, quality, or user satisfaction requirements.

4. Regarding the MDP elements

The basic strategy with RL to resolve scheduling problems is to define the MDP model by specifying the state space, action space, and reward function. The DRL algorithm is composed of these three fundamental elements. The state space is a collection of all state information about the cloud computing environment, including not only the resource requirements of tasks waiting to be scheduled but also the current resources available in the data center. Selecting an action means that the agent allocates the corresponding computational resource for the current task, and executing the action means that the task is executed on the specified resource. Therefore, the action space is a collection of discrete values, the elements of which are usually represented by the index of the server. In reinforcement learning, the agent is guided by reward signals to learn for optimization purposes. The setting of the reward function was the most significant variation in these trials. We will discuss the setting of the reward function in detail in Section 5.3.

5. Regarding the benchmark

The number of algorithms used as a benchmark in the study is illustrated in Fig. 6. According to the statistical data, Round Robin (RR) was used as the comparison algorithm in nine experiments, with Random being utilized in the following seven instances. The most popular reinforcement learning algorithm, DQN, was utilized as the comparison method in five research. Tetris was used four times, greedy twice, and the other algorithms each once. Since heuristic algorithms comprise the majority of these comparison algorithms, it seems that the scheduling technique based on RL/DRL is superior to some heuristic algorithms. A frequent baseline for reinforcement learning is DQN. The effect of various DRL algorithms based on the task scheduling environment has not yet been studied in any depth.

5.3. Relationship between scheduling objective and reward function

In the RL/DRL-based energy efficient task scheduling problem, the reward function is used to guide the DRL agent to learn a better task scheduling policy for higher long-term rewards to optimize energy consumption or other goals of cloud computing. The definition of the reward function is necessarily linked to the scheduling objective.

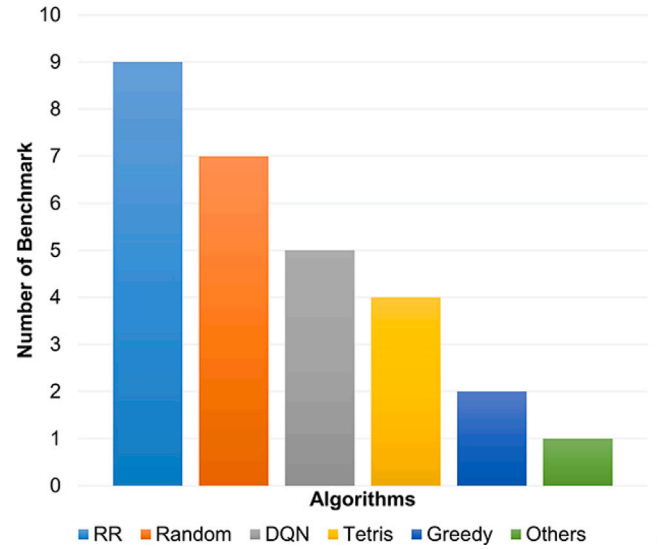


Fig. 6. Number of benchmarks in the literature.

As shown in Table 5, we summarize and compare the details in the scheduling objective and the reward function of the study based on energy efficient task scheduling with the DRL method. It can be noted that the reward function, in the majority of the research literature, is negatively linear or inversely proportional to the energy consumption value. The observed correlation relationship between energy consumption and reward function might be explained in this way: the scheduler agent receives a reward signal from the environment after executing an action (i.e. the task is executed on the scheduling server). The goal of the agent is to obtain the maximum cumulative reward based on the learned scheduling policy. However, we expect to minimize the total energy consumption value of the data center after the execution of the task. Therefore, the energy consumption value obtained after the execution of a task is set to a negative value, so that the agent receives a smaller reward value (a penalty signal) when the computing resource consumes more energy. Conversely, when the agent receives the maximum cumulative reward, it means that the energy consumption value is minimized.

This observation may support the hypothesis that for other scheduling objectives such as maximization parameters like throughput, resource utilization, etc., the reward function settings are usually positively correlated with these scheduling objectives. However, for minimization parameters such as task waiting time, task completion time, execution cost, energy consumption, etc., the reward function is negatively correlated with the scheduling objectives.

In addition, task scheduling optimization can be divided into single-objective and multi-objective based on the number of scheduling objectives. It is impractical to focus on a single energy usage objective while ignoring the cloud computing QoS or SLA in a real-world production setting. Therefore, most research focuses on multiple goals, such as minimizing energy consumption while keeping the deadline or budget constraints. The reward function is typically specified to an aggregated formulation of the scheduling objectives, and the weighting factor among these scheduling objectives should be adjusted to balance the power consumption and the other optimal objective.

Therefore, this makes it possible to create a paradigm for setting the reward function. Assuming that the scheduling objectives are obj_1, obj_2, obj_3 , and these three objectives are maximizing, minimizing, and maximizing parameters respectively, then the reward function in DRL is denoted as:

$$\begin{aligned}
 \text{Reward} &= \omega_1 * obj_1 - \omega_2 * obj_2 + \omega_3 * obj_3 \\
 \omega_1 &\in [0, 1]; \omega_2 \in [0, 1]; \omega_3 \in [0, 1]; \\
 &\& \omega_1 + \omega_2 + \omega_3 = 1
 \end{aligned} \tag{9}$$

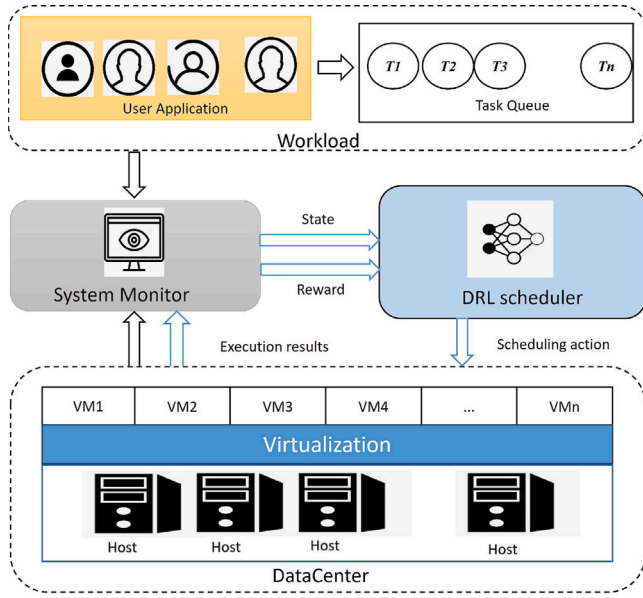


Fig. 7. System framework of DRL-based task scheduler.

$\omega_1, \omega_2, \omega_3$ are hyper-parameters as the weights of the three scheduling objectives, which can be adjusted according to the importance of the targets. In the formula, the value of obj may be the direct result of the calculation of the scheduling target [40], or it may be the result of the correlation of the target value. Zhao et al. [49] normalized each reward in a symmetrical $[-1, 1]$ space to stabilize the training process and make it a lot easier to debug. Chen et al. [63] regarded the average energy consumption of all completed jobs instead of total energy consumption as a reward. Lou et al. [38] defined the reward by dividing the average energy used at the current epoch by the average energy used at the previous epoch.

This approach of establishing the reward function under the scheduling objective is not limited to the study of scheduling for energy efficiency but can be generalized and validated for the scheduling of non-energy targeting task scheduling with the DRL algorithm. [17] proposed DRL-based scheduling agents to learn the scheduling policy for minimizing both the cost of using the cluster and the average job completion time. The episode reward formula was configured as a linear combination of average job duration and cost. [69] proposed a hierarchical framework to handle the resource allocation and power management problem in cloud computing systems. This work also defined the reward function through a joint consideration of power consumption, job latency, and reliability in the DRL-based global tier. The weight factor approach has also been utilized in bionics-based meta-heuristic task scheduling applications [70–72] for setting the fitness function to balance multiple scheduling objective optimizations.

5.4. General DRL-based scheduling framework

In this section, we present a general framework for a DRL-based task scheduling system, as depicted in Fig. 7. This framework offers a structured approach to developing and implementing efficient task scheduling algorithms using DRL methodologies. The proposed framework comprises several key components that collaboratively enable effective task scheduling: Workload, Data Center, System Monitor, and DRL Scheduler.

- **Workload:** Requests from different user applications are transformed into tasks with their respective requirements (resources, SLA) and placed in the task queue.

- **Data Center:** The cloud computing data center utilizes virtualization technology to partition hosts into heterogeneous virtual machines, which serve as the fundamental units for task execution.
- **System Monitor:** The system monitor module plays an environmental role in reinforcement learning. At each time step, it receives and encodes information concerning available resources, task characteristics as well as the current scheduling status. It will return the execution results as rewards to the scheduler once the task has been completed.
- **DRL Scheduler:** The DRL scheduler serves as the core component of the framework, responsible for learning and making decisions regarding task scheduling. This model comprises neural networks, such as deep Q-networks (DQNs) or policy gradient methods, which are trained to maximize a predefined reward function.
- **Task Scheduling Process:** In a scheduling batch, the system monitor captures the current state of the task scheduling system. The state representation serves as the input to the DRL scheduler. Functioning as an autonomous agent, the DRL scheduler employs the principles of the DRL algorithm to determine and execute scheduling actions, which involve task assignment to specific resources. After tasks are executed according to the scheduling policy, relevant execution results such as execution time, energy consumption, and resource utilization, are fed back to the system monitor module as rewards for the previous action. Meanwhile, the rewards are stored to experience replay buffer for learning. The scheduler employs DRL techniques to learn from the experience data, updating the neural network parameters to maximize the cumulative reward for an optimized scheduling policy.

The DRL-based task scheduling system framework allows the development of customized task scheduling algorithms adapted to a variety of scheduling objectives, environmental factors, and requirements.

6. Comparison of DRL framework, dataset for workload, structure of DNN

We conduct a comparison of the MDP elements across various DRL algorithms. Furthermore, there is a lack of literature investigating the implementation details of these algorithms in various application scenarios. The motivation of this section is to compare and summarize the key aspects of implementing DRL-based task scheduling algorithms in the context of artificial intelligence. The aim is to provide a more comprehensive reference for the algorithm implementation process. In this section, we present a detailed comparison of the deep learning frameworks, experimental datasets, and deep neural networks used in the implementation of the DRL algorithm, which also gives solutions to RQ5, RQ6, and RQ7. Specific aspects of comparison are shown in Table 6: algorithm type, workload dataset, algorithm implementation framework, DNN type, DNN structure and energy related results. We also produce a taxonomy based on the three dimensions and show them in Fig. 8 to allow readers rapid access to the research findings.

6.1. Framework of deep reinforcement learning

The rise of deep learning has been fueled by several factors including the availability of large datasets, abundant computing resources, and the development of accessible deep learning frameworks. These frameworks, such as TensorFlow and PyTorch, offer an interface for designing, training, and testing deep learning models, as well as providing blocks that can be used to build different kinds of deep neural networks. As shown in Table 6, both TensorFlow and PyTorch have been employed as experimental settings in several DRL-based studies. PyTorch was utilized in the research of [41,49], while TensorFlow was used as the experiment environment in [3,34,36,40,42,63,65].

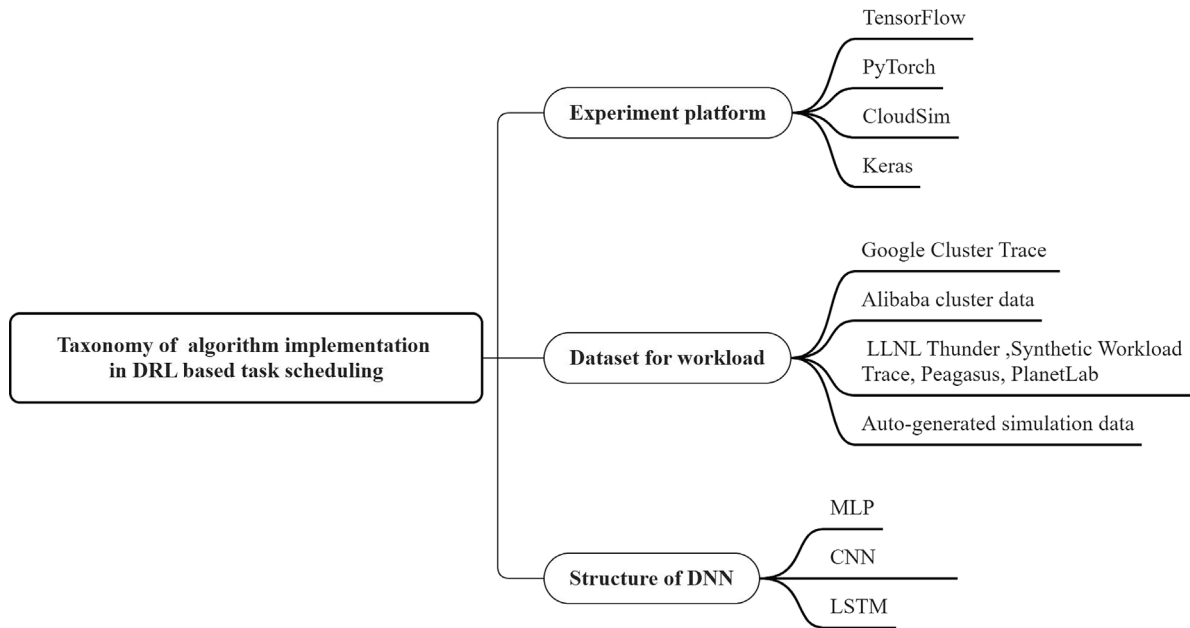


Fig. 8. Taxonomy of algorithm implementation in DRL based task scheduling.

Keras, which is built on top of TensorFlow and PyTorch, offers a more universal experience and speeds up development. In [64], the CloudSim toolkit was combined with the Keras library to implement new modules for simulating their workflow scheduling framework and interacting with the deep learning algorithms.

PyTorch, launched in 2016, is a library that achieves a balance between usability and speed [73]. It enables fast, flexible experimentation and efficient production through a user-friendly front-end, distributed training, and an ecosystem of tools and libraries. TensorFlow, on the other hand, was first released in 2015 and its goal is to enable the execution of any machine learning algorithm across a range of platforms [74]. TensorFlow 2.0, released in 2019, offers a more user-friendly interface and is effective in terms of performance.

One of the biggest differences between the two frameworks is the way of creating and running computational graphs. In PyTorch, the graph architecture is dynamic, while in TensorFlow, the graph architecture is static. Various studies have compared the two frameworks based on user-friendliness, readily available documentation, ease of integration, overall training time, overall accuracy, and execution time during evaluation. For instance, Jain et al. [75] provided a detailed comparison between the two neural network libraries using the same input data format, hardware requirements, and neural network architecture. The comparison reveals that TensorFlow 2.0 is less user-friendly and not easier to integrate than PyTorch. In comparison to PyTorch, TensorFlow performs better on CPUs although PyTorch performs better on GPUs. In another comparison made by [76], TensorFlow offers more overall options at the sacrifice of user-friendliness, whereas PyTorch is more beginner-friendly. PyTorch outperforms TensorFlow in terms of training and running, despite having lower accuracy and nearly equal training paths.

While CloudSim [77] is a robust toolkit widely used for modeling and simulating cloud environments, offering customizable and extended simulation components. However, due to its Java-based architecture, it faces certain limitations when it comes to implementing deep learning techniques. On the other hand, TensorFlow and PyTorch, being Python-based frameworks, provide extensive code libraries and frameworks for machine learning. Consequently, they have emerged as the most popular development frameworks for implementing DRL algorithms.

In conclusion, both TensorFlow and PyTorch have their strengths and weaknesses. The choice of library may have some influence on

design and user experience, but they have little impact on system performance during training.

6.2. Dataset for workload

The performance of DRL scheduling algorithms heavily relies on the availability and quality of training data. The most fundamental training data for task scheduling problems are data on task sets and computational resources [62]. Some recent studies, such as [16,49,62], generate experiment data automatically, while most studies use real cluster tracking datasets. According to the survey data in Table 6, the two most representative cloud computing traces are the Google Cluster Trace [3,34,35,38,41,42,48,63,78] and the Alibaba cluster-trace [40,65,67]. Other real datasets used in the literature include LLNL Thunder [36], Synthetic Workload Trace [34], Peagasus workflow [64], and PlanetLab Virtualized research datasets [65].

The Google Cluster Trace is considered the most popular trace and a benchmark dataset, which is publicly accessible [78]. Google released a trace of a cluster of 11k machines in May 2011, which contains cell information for about 29 days [79]. The trace data contains about six tables: Machine Events Table, Machine Attribute Table, Jobs Events Table, Task Events Table, Task Constraints Table, and Task Resource Usage Table. A single usage trace typically describes several days of the workload on one of the computing cells (sets of machines). A trace is made up of several datasets. A dataset contains a single table, indexed by a primary key that typically includes a timestamp. Each dataset is packaged as a set of one or more files, each provided in a compressed Comma-Separated Values(CSV) format. Researchers can perform data analysis and extraction based on tracing to get profiles about cloud workload, resources, and machine usage for applications on cloud computing or cluster management. Synthetic Workload Trace (SWT) is proposed based on characteristics of Google Trace with Gaussian noise integrated to generalize regular workloads [34]. SWT can detect the typical peak and trough changes that occur alternately and are utilized as a control group of workload characteristics in their experiment.

Another popular dataset is Alibaba cluster data. The well-known cloud service provider, Alibaba Cloud, has previously made some Alibaba traces public. Five different versions of traces have been made available so far:

Table 6

Comparison of DRL framework, dataset for workload, structure of DNN.

Author	DRL algorithm	Dataset	DRL framework	DNN type	DNN structure	Energy related results
[48]	PPO-Clip	Google cluster traces	N/A	MLP	Small-scale:3 layers (64,32,16), Medium-scale: 3 layers (64,32,16), Large-scale: 3 layers (128,64,16)	The proposed algorithm compared with the Tetris: 1. 80,000 tasks: 23.8% Energy reduction 2. 120,000 tasks: 27.5% Energy reduction 3. 240,000 tasks: 35.4% Energy reduction
[16]	Q-learning	Simulation data	CloudSim	N/A	N/A	The proposed QEEC is the most energy-efficient compared with the RANDOM, FAIR, GREEDY.
[3]	DQN	Google cluster traces	Python, TensorFlow	N/A	N/A	The proposed DRL-Cloud improves 320% energy cost efficiency compared with FERPTS, it achieves up to 144% runtime reduction, 218% energy cost efficiency improvement and 249% lower reject rate compared to RR.
[41]	Quantile Regression DQN	Google cluster traces	Python3, Pytorch	N/A	N/A	The QR-DQN has super energy cost efficiency and low task reject rate to RR. The average energy cost saving is about 200% compared with RR.
[49]	PPO	Stochastic Simulation Data	Python3.8, OpenAI Gym and PyTorch	MLP	4 layers(256, 128, 64, 32)	The proposed DRL policy reduce energy consumption by 26.87%, 34.61%, 8.40% compared to RR, MRLUF, and DQN, respectively.
[35]	Soft Actor–Critic	Google cluster traces data 2019	Random, DQN	MLP	Soft Q-network: 2 hidden layers (400,300)	The proposed WSS reduces energy consumption by nearly 25% without server overload compared with tDQN.
[62]	DQN	Simulation Data	N/A	MLP	2 layer MLP (20,20)	Compared with Round Robin and Earliest, the proposed algorithm achieves about 35%–40% higher success rate, less energy cost, only about 60%–70% of the baselines.
[36]	Parameter-ized Action Space DQN	A real workload trace “LLNL Thunder”	TensorFlow	MLP	Q network: 2 fully-connected layer (512,64) ReLu() Policy network: 3 fully-connected layer (512,1024,32) tanh()	The proposed PADQN algorithm can save up to 15% and 10% energy consumption compared with the baseline and joint optimization approaches respectively.
[34]	DQN	Google Trace, Synthetic Workload Trace	Python 3 TensorFlow	MLP	2 hidden layers (30,15)	The proposed ADRL shows an energy improvement over RR by 246.22% and performs best on both the energy consumption and average response time compared with Hierarchical DRL, DRL-Cloud and MO-DQN.
[63]	A3C	Google cluster trace	TensorFlow 1.4.0	MLP	2 fully-connected hidden layers(200,100)	The proposed method can save around 9% of the average energy consumption compared to the RR while achieving excellent QoS (normalized average job latency and job dismissing rate).
[38]	DQN	Google cluster trace	N/A	MLP	Q network: 2 fully-connected layers (30,10)	The proposed ODTs has the best energy efficiency and saves 14% of energy compared with the BF baseline, achieving substantial improvement for all average SLAV.
[64]	PPO	Peagatus workflow framework	Cloudsim, Keras	MLP	Actor1: 2 fully-connected hidden layers (100), Critic: 2 fully-connected hidden layers (100), Actor2: 4 fully-connected hidden layers (100)	The proposed DRL framework consumes 32%, 56% and 75% less energy compared to EDA, EFT and Random algorithms, respectively. An improvement of 11% and 47% over the execution times of Random and EES is achieved.
[40]	Policy Gradient with baseline	Alibaba Cluster Data V2017	TensorFlow	N/A	N/A	The proposed DeepEnergyJS achieves between 1% to 22% decrease in energy consumption compared with the baselines.
[65]	PPO	Alibaba Cluster Data V2018	TensorFlow	N/A	N/A	The proposed DeepEnergyJSV2.0 achieves better energy efficiency than DeepEnergyJS by about 7.814%.
[66]	Q-learning	PlanetLab Virtualized research datasets	CloudSim	N/A	N/A	The proposed mechanism allocates resources with Power Usage Effectiveness (PUE) value in an efficient range from 1.79 to 1.96 having reasonable resource utilization (above 50%) for the cloud data center.
[42]	DQN	Google cluster traces	Python TensorFlow	N/A	N/A	The proposed DQN algorithm consumes less energy than the baseline algorithm when the number of tasks exceeds 200.

- cluster-trace-v2017 includes about 1300 machines in a period of 12 h.
- cluster-trace-v2018 includes about 4000 machines in a period of 8 days which also contains the DAG information of our production batch workloads.
- cluster-trace-gpu-v2020 includes over 6500 GPUs (on 1800 machines) in 2 months.
- cluster-trace-microservices-v2021 contains 20000+ microservices in 12 h. The traces the first released to introduce the runtime metrics of microservices in the production cluster.
- cluster-trace-microarchitecture-v2022 first provides AMTrace (Alibaba Microarchitecture Trace).

[40,67] used workload data extracted from the cluster-trace-v2017 in their experiments including dependent tasks and DAG tasks. Cluster-trace-v2017 also consists of six tables. Machines are described by two tables: the machine events table and the machine resource utilization table. Batch workloads are described by these tables: Instance table and Task table. Online services are described by tables: service instance event and service instance usage. Each data table is given in CSV format, using Unix-style line endings. The CSV files have no header. The schema for all tables is also given in CSV format in a file called schema.csv.

[78] performed a case study on Google trace and cluster-trace-v2018. The trace statistics and data structure are analyzed and compared based on the overall view. The results implied that the workloads have diversity between the two traces from the top architecture level to the bottom machine level. Despite the differences between the two traces, the fact that these trace data originate from a real production environment means that they may accurately depict the randomness and fluctuation of actual application requests as well as the current state of the computing resources.

In summary, real-world tracking not only allows people in related fields to understand contemporary data centers better but also gives researchers production data to back up their theories. The large real datasets can provide a good source of training data for the DRL algorithm in task scheduling and resource provision. By simulating workloads under real-world scenarios, researchers or practitioners can produce results that are more likely to be adopted by businesses.

6.3. Structure of deep neural network

DNNs are used as a key component in the training process of DRL algorithms. The superior performance of DNNs comes from their ability to extract high-level features from a large amount of raw data to obtain an effective representation of an input space [80]. In this section, applications for task scheduling with different structures of DNNs are reviewed, and then some common principles for selecting the hyperparameters are discussed by analyzing the neural network architecture of the agent for the DRL algorithm.

A neural network consists of numerous artificial neurons interconnected in various ways. The fundamental neural network model comprises three layers: the input layer, the hidden layer, and the output layer. Each layer contains multiple neurons, and the connections between the layers are fully connected, meaning that every neuron in the preceding layer is linked to all neurons in the subsequent layer. Typically, the output of the fully connected layer is passed through an activation function to capture complex relationships. DNN can be seen as a neural network with multiple hidden layers, as depicted in Fig. 9. DNNs are also referred to as Multi-Layer Perceptions (MLPs).

Another sort of neural network is the Convolutional Neural Network (CNN) [81], which is highly effective in various image-related tasks. In addition to the input and output layers, CNNs typically incorporate three key components: the convolutional layer, the pooling layer, and the fully connected layer. The convolutional layer plays a crucial role in extracting essential features from images. The pooling

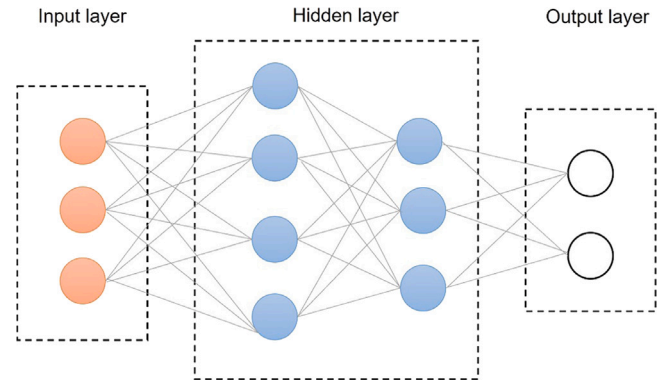


Fig. 9. Structure of DNN.

layer performs downsampling operations, reducing the spatial dimensions while retaining important information. Finally, the fully connected layer classifies the extracted features.

Additionally, Long Short-Term Memory (LSTM) [82] is a specific Recurrent Neural Network (RNN) architecture. LSTMs are well-suited for sequential data analysis, such as natural language processing, speech recognition, and time series forecasting. LSTMs incorporate specialized memory cells that can retain and update information over extended periods, enabling them to capture long-term dependencies in sequential data effectively.

Both CNNs and LSTMs are variations of DNNs. The features of the relevant literature are reviewed below based on the three neural network structures.

- MLP structure: It is apparent from Table 6 that MLP is the most commonly used neural network model for task scheduling problems. Several studies have utilized MLP networks with two fully connected layers, including [34,35,38,62,63]. For instance, in [62], two fully connected layers with sizes of (512, 64) were used as the Q-network and policy network, respectively. In [48], three fully connected layers of the PPO algorithm were leveraged in small, medium, and large-scale heterogeneous experiments. Additionally, Zhao et al. [49] trained their PPO-based task scheduling algorithm with the structure of four fully connected layers. In the Actor-Critic based scheduling framework with PPO proposed by [64], two hidden layers were used in the critic network and first actor network, and four hidden layers were employed for the second actor network.

The majority of agents in task scheduling problems use neural networks with a maximum of four hidden layers, and the most commonly employed architectures contain two or three fully connected networks in addition to the input and output layers. The state in the MDP model of the task scheduling system is represented by a feature tensor that combines the current task information and the resource state. The neural network receives this tensor as input directly, and after several fully connected layers, it generates an output that is also in vector format. The output of the neural network typically indicates the action to be performed by the agent, in other words, the machine to be assigned for the current task.

- CNN structure: There is another proposal in the literature that utilizes image information to represent the state space and action space. In this approach, images are utilized to represent both the current resource state of the system and the requested task state.

Mao et al. [83] first applied DRL to the resource scheduling problem, representing the current allocation of cluster resources and the resource profiles of jobs waiting to be scheduled as distinct images. They trained their DeepRM model using a variant of the REINFORCE algorithm, and the results showed that it is feasible to apply DRL techniques to job scheduling. Chen et al. [84] extended DeepRM to multiple server clusters and built a multi-layer CNN to extract features from the state matrix. The model outperformed heuristic methods such as Shortest Job First and Packer. Han et al. [85] constructed an RL environment based on a disjunctive graph and utilized a deep CNN to approximate the state-action value. The manufacturing states were expressed as multi-channel images and input into the network. In the experiments, a dueling double DQN with prioritized replay was used, and the optimal network structure comprised four convolutional layers and a fully connected layer. Liang et al. [86] proposed an A2C Scheduler based on the A2C algorithm to address the job scheduling problem in data centers. They trained the agent with different neural networks, including a fully connected layer and CNN. Different CNN structures were explored, and convergence under various circumstances was evaluated.

- **LSTM structure:** Several studies have also explored the use of LSTM networks to address the task scheduling problem. LSTMs are popular in sequence prediction tasks, as they can effectively capture long-term dependencies in time-series data. For example, Liu et al. [69] proposed a hierarchical framework for resource allocation and power management, in which the LSTM neural network was used to improve the accuracy of workload prediction at the local server. Similarly, Pandey et al. [87] presented an energy-efficient strategy based on LSTM-DQN for big data in cloud environments, using LSTM for load prediction when abnormal data characteristics were detected. Combining LSTM with DRL is another approach. Rjoub et al. [88] employed an LSTM unit as the first layer of a DRL model to capture long-term dependencies in the data for addressing the time-dependent task scheduling problem. Their experiments showed that the proposed DRL-LSTM model can significantly reduce the CPU and RAM usage cost compared to baseline methods such as Round Robin (RR), Shortest Job First (SJF), and Particle Swarm Optimization (PSO).

From the analysis of the characteristics of the surveyed works, the following observations about the parameters of neural networks are highlighted:

There is no clear theoretical derivation that demonstrates how many layers are appropriate; the number of hidden layers depends on the situation. The DNNs tend to offer a richer level of abstraction and dimensionality reduction when they consist of more than two hidden layers. However, more layers are not necessarily better. Each additional fully connected layer in a DNN structure significantly increases the number of weight parameters. This expansion not only extends the training time but also introduces the risk of getting trapped in local optima. In some cases, the performance of a deep network can even be inferior to that of shallower networks when the available training data is limited.

In terms of the number of neurons in each layer, generally, the number of input dimensions can be used as a basic guideline to determine how many neurons should be set in the first layer. It is beneficial to have neurons in the power of 2, as it helps the computation of the network to be faster [89].

Moreover, it is worth noting that some research [85,86] did not use a pooling layer behind the convolutional layer in their neural network, which deviates from the typical CNN structure. Each pixel within the adopted state expression signifies a specific operation, and the inclusion of a pooling layer removes certain pixels, leading to an incomplete state representation. Therefore, the algorithm may not converge or

produce poor results with a pooling layer. Currently, no research has yet explored the impact of structural differences in neural networks on the performance of the DRL algorithm in task scheduling problems.

7. Research gaps, open challenges and future directions

7.1. Research gaps

Based on our investigation into DRL-based task scheduling optimization in cloud computing, we have identified certain research gaps. Specifically, there is a lack of thorough comparison of performance between RL-based scheduling techniques, heuristic scheduling, and meta-heuristic algorithms in the literature. Moreover, there has been no research conducted to evaluate the effectiveness of different DRL algorithms for the task scheduling problem, such as comparing policy-based and value-based learning or exploring learning strategies that combine the two. Additionally, it is imperative to investigate the impact of structural differences in neural networks on the performance of the same DRL algorithm in task scheduling problems.

7.2. Open challenges and future directions

DRL-based scheduling in cloud computing has made significant progress. However, there are still many issues in this area that need to be solved. Based on current research, we discuss a series of open research challenges and potential future directions across five distinct dimensions: the inherent characteristics of DRL technology, evolving computing paradigms, innovative virtualization technologies, the integration of renewable resources, and the critical domains of safety, security and privacy.

7.2.1. Limitations of DRL technology

DRL is a powerful approach for handling complex decision-making processes. Studies have demonstrated that DRL-based algorithms can solve task scheduling problems that involve multiple objectives and constraints. But it also comes with certain limitations that can impact its applicability and effectiveness in various contexts.

- **High Computational Demands:** Training deep neural networks, which are common components of DRL algorithms, is time-consuming and requires significant processing power and memory. This may limit the scalability and real-time applicability of DRL in resource-constrained settings. Therefore, how to ensure the scalability of the DRL approach is an urgent challenge.
- **Reward Engineering:** Designing appropriate reward functions is crucial for successful DRL training. Poorly designed rewards can lead to suboptimal or undesirable behaviors in the learned policies. It takes some effort to create a reward function that accurately expresses the optimal goal.
- **Instability and Convergence:** DRL training may suffer from convergence issues or instability, leading to slow learning or policies that fail to improve over time. It might be difficult to provide consistent convergence, especially in complicated contexts.
- **Hyperparameter Sensitivity:** Hyperparameter tuning of deep neural networks to obtain better algorithms remains a challenge. Choosing the right hyperparameters is an intensive and iterative process, it takes an abundance of domain knowledge to carefully design the network and fine-tune the hyperparameters in DRL training.
- **Generalization to Unseen Scenarios:** An algorithm trained in a simulated environment may cause the policy to perform poorly in practical application due to the unpredictable fluctuation of realistic workloads or resources. Adapting learned policies to new or unseen situations, known as transfer learning, may be challenging.

7.2.2. Context of the cloud computing paradigm shift

Cloud computing has been a mature computing paradigm. With the boom of IoT and more devices joining the Internet to request computing resources, fog computing [90], as an emerging computing technology, is placed between the IoT and cloud layers with the goal of better management of resources, proper data pre-processing, shorter time delays in data transmission, and reduced network graphics overhead. In addition, edge computing extends computing resources to the edge of the network, thus enabling data to be processed and analyzed closer to the generating source [91]. This enables an integrated cloud–edge paradigm to meet the QoS requirements of latency-sensitive and bandwidth-hungry IoT applications while taking advantage of public cloud platforms to provide computing resources [64]. The change in computing paradigm has brought new characteristics that challenge the traditional centralized-based scheduling approach:

- **Heterogeneous Resources:** Edge and fog nodes come with different computational capacities, memory, and storage capabilities [92]. Heterogeneous resources and heterogeneous nodes (end devices, edge servers, mobile devices) make resource management and scheduling complicated.
- **Dynamic Workloads:** Edge and fog environments experience dynamic workloads due to varying numbers of connected devices and changing task requirements. These applications are diverse, with some being real-time applications and others being delay-tolerant applications [93]. Different applications have various QoS requirements and SLAs.
- **Energy Efficiency:** IoT devices and edge nodes often operate under strict energy constraints.
- **Network Bandwidth:** The limited bandwidth between edge nodes and the centralized cloud or other nodes can lead to congestion and increased latency.
- **Distributed Nature:** Traditional centralized scheduling approaches may not be suitable for edge and fog computing. Distributed decision-making and coordination among nodes are promising approaches for effective task scheduling.

DRL technology already has solutions to these challenges due to its strong perceiving and decision-making abilities, such as dynamic workload prediction based on DRL [87,88], centralized training and distributed execution based multi-agent DRL [94], multi-objective and multi-constraint scheduling based on DRL [64]. It has great potential in the multi-layer computing environment. However, the majority of their research is based on simulation experiments, it would take plenty of work to adapt them to a real environment. Additionally, compared with cloud computing servers, fog computing, and edge computing resources are limited in terms of computation, storage, and energy [95]. Therefore, running DRL-based algorithms on these devices is a challenging task, and proper lightweight algorithm design is required to meet this challenge.

7.2.3. Context of container-based virtualization

As an emerging virtualization solution, containers have gained tremendous popularity in recent years and are replacing traditional virtual machines due to many promising properties such as shared host OS, quick launch time, portability, scalability, and fast deployment [96]. In this context, the application architecture is converted from monolithic applications such as containers to microservice or serverless architecture.

- **Microservice Architecture:** It has significantly reduced the costs and complexity of application development and deployment. Nonetheless, the growing number of components and dynamic inter-dependencies between microservices raises the problem of load distribution and resource management at the infrastructure level [97]. One promising approach is taking advantage of advanced DRL-based approaches to establish comprehensive analysis models for microservices.

- **Serverless Architecture:** It is designed to perform certain user-defined functionalities. The time overhead of function initialization is currently the major performance bottleneck in function invocation. How to optimize the invocation of function chains is also a crucial research question, especially under limited computational resources and battery capacity.
- **Custom Container Scheduler:** The mainstream container orchestration platforms, such as Kubernetes and Swarm, are designed to manage the deployment of containerized applications in large-scale clusters [98]. However, the built-in schedulers of these orchestration systems are unable to accommodate all application circumstances. There have been some custom schedulers [99,100] based on K8s for specific applications, but DRL-based scheduler research has not been explored. A scheduler with learning ability is crucial in complex environments, therefore the potentiality of DRL in container scheduling is worth being explored.

7.2.4. Renewable energy usage

Minimizing the use of fossil fuels in the power supply system of the data center can reduce carbon emissions effectively. Renewable energy sources (like solar, wind, or hydroelectric energy) have been recognized as clean and sustainable alternatives to fossil-fuel-based brown energy [52]. Currently, integrating renewable energy into cloud computing requires dealing with several practical and technical hurdles.

- **Intermittency and Variability:** Renewable energy sources such as solar and wind are inherently intermittent and variable [49]. Cloud computing requires a stable and consistent power supply to ensure uninterrupted operations. Integrating these fluctuating energy sources into data centers requires sophisticated energy storage and management systems to bridge the gap between energy generation and consumption.
- **Grid Integration and Stability:** Integrating renewable energy sources into the power grid requires careful coordination to maintain grid stability. Cloud data centers may need to interact with the grid to import or export energy based on availability [101]. Ensuring a seamless interaction between the data center and the grid without affecting the quality of service is a significant challenge.
- **Energy-Efficient Data Center Design:** Data centers need to be designed and optimized for energy efficiency to minimize overall energy consumption [102]. This includes considerations such as server and cooling system design, use of energy-efficient hardware, and data center layout to reduce energy waste.

There have been some pioneering research activities to match demand with renewable energy supply for cloud data centers in a simulated environment [49,52,103]. Yet the practical solution may involve a combination of technological innovations, energy storage advancements, and policy advocacy to fully realize the potential of renewable energy in cloud computing.

7.2.5. Safety, security and privacy

In this section, we discuss safety, security, and privacy issues and possible solutions in DRL technology and multilayer computing paradigms, respectively.

(1) DRL technology

- **Safety:** DRL lacks intrinsic mechanisms for ensuring safety, and learned policies may exhibit unsafe behaviors in critical or uncertain situations. Implement exploration strategies that ensure the agent explores the environment while minimizing the risk of dangerous or uncharted actions. Techniques like optimism in the face of uncertainty (OFU) can guide exploration towards safe regions [104].

- **Security:** Security is a crucial concern, as vulnerabilities and attacks can compromise the integrity, availability, and performance of the learning process, leading to potential misuse or malicious manipulation. Developing DRL algorithms that explicitly consider security and adversarial scenarios in their learning objectives can make the model more resistant to adversarial attempts [105].
- **Privacy:** Ensuring privacy in DRL based approach is a big challenge due to the sharing of sensitive data during training and the potential for unintended information leakage. Differential privacy, which introduces noise or randomness to the learning process to protect individual data points [106]. Federated learning is a privacy-preserving technique that distributes the learning process across multiple devices or agents while keeping the data localized [107]. In reality, it is possible to select the best privacy protection technology as required or even combine different types of privacy protection to get a better level of privacy protection.

(2) Multi-layer computing paradigms

- **Safety:** Edge and fog computing environments often consist of heterogeneous and distributed devices, some of which may be untrusted or compromised. Ensuring the safety and reliability of computation and data processing in such environments is a challenge [108].
- **Security:** It is challenging for fog devices, IoT, and edge computing nodes to execute proper security solutions due to limited computing capacity. Establishing secure authentication and authorization mechanisms in a decentralized environment is challenging, especially when devices have varying levels of trust [95].
- **Privacy:** Edge and fog computing involves processing data closer to its source, raising concerns about data privacy. Ensuring proper data anonymization, encryption, and access control is crucial.

Addressing these challenges requires a holistic approach that takes into account technologies such as security and trust, privacy protection, and the development of solutions suited to the specific characteristics of edge and fog computing environments.

8. Conclusion

Task scheduling in cloud computing is a significant and emergent research area. In this paper, we reviewed the literature on energy efficient task scheduling with DRL methods. The energy consumption models in relevant studies were compared and classified. A thorough comparative analysis of energy-efficient DRL-based task scheduling was conducted, considering various aspects such as RL/DRL algorithms, task types, scheduling objectives, state spaces, action spaces, reward functions, and baseline. Additionally, we explained the fundamental guidelines for setting the reward function and proposed a general framework of a DRL-based task scheduler. Based on the reviewed literature, the experimental framework of the DRL algorithms, the experimental dataset, and the structure of the DNN were compared and analyzed. Finally, we discussed the research gaps, open challenges and future directions from five different perspectives. We hope that this in-depth review will be a valuable resource for researchers in the subject and serve as a foundation for further exploration and experimental effort.

CRedit authorship contribution statement

Huanhuan Hou: Conceptualization, Methodology, Investigation, Formal analysis, Writing – original draft. **Siti Nuraishah Agos Jawaddi:** Writing – review & editing. **Azlan Ismail:** Supervision, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no competing interests.

Data availability

Data will be made available on request.

Acknowledgments

Huanhuan Hou acknowledges the support of a Research Grant, 2019LG04, funded by the Science Foundation of Taiyuan Institute of Technology. Furthermore, Azlan Ismail acknowledges the support of the Fundamental Research Grant Scheme, FRGS/1/2018/ICT01/UITM/02/3, funded by Ministry of Education Malaysia.

References

- [1] H. Hu, Z. Li, H. Hu, J. Chen, J. Ge, C. Li, V. Chang, Multi-objective scheduling for scientific workflow in multicloud environment, *J. Netw. Comput. Appl.* 114 (2018) 108–122.
- [2] H. Lai, H. Liao, J. Šaparauskas, A. Banaitis, F.A. Ferreira, A. Al-Barakati, Sustainable cloud service provider development by a Z-number-based DNMA method with gini-coefficient-based weight determination, *Sustainability* 12 (8) (2020) 3410.
- [3] M. Cheng, J. Li, S. Nazarian, DRL-cloud: Deep reinforcement learning-based resource provisioning and task scheduling for cloud service providers, in: 2018 23rd Asia and South Pacific Design Automation Conference (ASP-DAC), IEEE, 2018, pp. 129–134.
- [4] R. Medara, R.S. Singh, Energy efficient and reliability aware workflow task scheduling in cloud environment, *Wirel. Pers. Commun.* 119 (2) (2021) 1301–1320.
- [5] M. Masdari, F. Salehi, M. Jalali, M. Bidaki, A survey of PSO-based scheduling algorithms in cloud computing, *J. Netw. Syst. Manage.* 25 (1) (2017) 122–158.
- [6] M. Kumar, S.C. Sharma, S. Goel, S.K. Mishra, A. Husain, Autonomic cloud resource provisioning and scheduling using meta-heuristic algorithm, *Neural Comput. Appl.* 32 (24) (2020) 18285–18303.
- [7] M. Hosseinzadeh, M.Y. Ghafour, H.K. Hama, B. Vo, A. Khoshnevis, Multi-objective task and workflow scheduling approaches in cloud computing: a comprehensive review, *J. Grid Comput.* 18 (3) (2020) 327–356.
- [8] A. Amini Motlagh, A. Movaghar, A.M. Rahmani, Task scheduling mechanisms in cloud computing: A systematic review, *Int. J. Commun. Syst.* 33 (6) (2020) e4302.
- [9] M. Xu, W. Tian, R. Buyya, A survey on load balancing algorithms for virtual machines placement in cloud computing, *Concurr. Comput.: Pract. Exper.* 29 (12) (2017) e4123.
- [10] M. Kumar, S.C. Sharma, A. Goel, S.P. Singh, A comprehensive survey for scheduling techniques in cloud computing, *J. Netw. Comput. Appl.* 143 (2019) 1–33.
- [11] S. Ilager, R. Muralidhar, R. Buyya, Artificial intelligence (ai)-centric management of resources in modern distributed computing systems, in: 2020 IEEE Cloud Summit, IEEE, 2020, pp. 1–10.
- [12] G. Zhou, W. Tian, R. Buyya, Deep reinforcement learning-based methods for resource scheduling in cloud computing: A review and future directions, 2021, arXiv preprint arXiv:2105.04086.
- [13] Y. Shang, J. Li, M. Qin, Q. Yang, Deep reinforcement learning-based task scheduling in heterogeneous MEC networks, in: 2022 IEEE 95th Vehicular Technology Conference (VTC2022-Spring), IEEE, 2022, pp. 1–6.
- [14] A. Asghari, M.K. Sohrabi, F. Yaghmaee, Task scheduling, resource provisioning, and load balancing on scientific workflows using parallel SARSA reinforcement learning agents and genetic algorithm, *J. Supercomput.* 77 (3) (2021) 2800–2828.
- [15] T. Dong, F. Xue, C. Xiao, J. Li, Task scheduling based on deep reinforcement learning in a cloud manufacturing environment, *Concurr. Comput.: Pract. Exper.* 32 (11) (2020) e5654.
- [16] D. Ding, X. Fan, Y. Zhao, K. Kang, Q. Yin, J. Zeng, Q-learning based dynamic task scheduling for energy-efficient cloud computing, *Future Gener. Comput. Syst.* 108 (2020) 361–371.
- [17] M.T. Islam, S. Karunasekera, R. Buyya, Performance and cost-efficient spark job scheduling based on deep reinforcement learning in cloud computing environments, *IEEE Trans. Parallel Distrib. Syst.* 33 (7) (2021) 1695–1710.
- [18] Z. Hu, J. Tu, B. Li, Spear: Optimized dependency-aware task scheduling with deep reinforcement learning, in: 2019 IEEE 39th International Conference on Distributed Computing Systems (ICDCS), IEEE, 2019, pp. 2037–2046.

- [19] Z. Tong, H. Chen, X. Deng, K. Li, K. Li, A scheduling scheme in the cloud computing environment using deep Q-learning, *Inform. Sci.* 512 (2020) 1170–1191.
- [20] T. Kaur, I. Chana, Energy efficiency techniques in cloud computing: A survey and taxonomy, *ACM Comput. Surv. (CSUR)* 48 (2) (2015) 1–46.
- [21] S.S. Gill, R. Buyya, A taxonomy and future directions for sustainable cloud computing: 360 degree view, *ACM Comput. Surv.* 51 (5) (2018) 1–33.
- [22] R. Ghafari, F.H. Kabutarkhani, N. Mansouri, Task scheduling algorithms for energy optimization in cloud environment: a comprehensive review, *Cluster Comput.* 25 (2) (2022) 1035–1093.
- [23] S.S. Panwar, M.M.S. Rauthan, V. Barthwal, A systematic review on effective energy utilization management strategies in cloud data centers, *J. Cloud Comput.* 11 (1) (2022) 1–29.
- [24] C. Shyalika, T. Silva, A. Karunananda, Reinforcement learning in dynamic task scheduling: A review, *SN Comput. Sci.* 1 (6) (2020) 1–17.
- [25] J.H. Joloudari, R. Alizadehsani, I. Nodehi, S. Mojriani, F. Fazl, S.K. Shirkharkolaie, H. Kabir, R.-S. Tan, U.R. Acharya, Resource allocation optimization using artificial intelligence methods in various computing paradigms: A review, 2022, arXiv preprint arXiv:2203.12315.
- [26] Y. Garf, D.A. Monge, E. Pacini, C. Mateos, C.G. Garino, Reinforcement learning-based application autoscaling in the cloud: A survey, *Eng. Appl. Artif. Intell.* 102 (2021) 104288.
- [27] S. Bharany, S. Sharma, O.I. Khalaf, G.M. Abdulsahib, A.S. Al Humaimedy, T.H. Aldhyani, M. Maashi, H. Alkahtani, A systematic survey on energy-efficient techniques in sustainable cloud computing, *Sustainability* 14 (10) (2022) 6256.
- [28] A. Katal, S. Dahiya, T. Choudhury, Energy efficiency in cloud computing data centers: a survey on software technologies, *Cluster Comput.* 26 (3) (2023) 1845–1875.
- [29] Q. Zhou, M. Xu, S.S. Gill, C. Gao, W. Tian, C. Xu, R. Buyya, Energy efficient algorithms based on VM consolidation for cloud computing: comparisons and evaluations, in: 2020 20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGRID), IEEE, 2020, pp. 489–498.
- [30] A. Ismail, Energy-driven cloud simulation: existing surveys, simulation supports, impacts and challenges, *Cluster Comput.* 23 (4) (2020) 3039–3055.
- [31] A. Arunarani, D. Manjula, V. Sugumaran, Task scheduling techniques in cloud computing: A literature survey, *Future Gener. Comput. Syst.* 91 (2019) 407–415.
- [32] R. Garg, M. Mittal, L.H. Son, Reliability and energy efficient workflow scheduling in cloud environment, *Cluster Comput.* 22 (4) (2019) 1283–1297.
- [33] R. Medara, R.S. Singh, M. Sompalli, Energy and cost aware workflow scheduling in clouds with deadline constraint, *Concurr. Comput.: Pract. Exper.* (2022) e6922.
- [34] K.X. Kang, D. Ding, H.M. Xie, Q. Yin, J. Zeng, Adaptive DRL-based task scheduling for energy-efficient cloud computing, *IEEE Trans. Netw. Serv. Manag.* (2021).
- [35] P. Song, C. Chi, K. Ji, Z. Liu, F. Zhang, S. Zhang, D. Qiu, X. Wan, A deep reinforcement learning-based task scheduling algorithm for energy efficiency in data centers, in: 2021 International Conference on Computer Communications and Networks (ICCCN), IEEE, 2021, pp. 1–9.
- [36] Y. Ran, H. Hu, Y. Wen, X. Zhou, Optimizing energy efficiency for data center via parameterized deep reinforcement learning, *IEEE Trans. Serv. Comput.* (2022).
- [37] A. Marahatta, Q. Xin, C. Chi, F. Zhang, Z. Liu, PEFS: AI-driven prediction based energy-aware fault-tolerant scheduling scheme for cloud data center, *IEEE Trans. Sustain. Comput.* 6 (4) (2020) 655–666.
- [38] J. Lou, Z. Tang, W. Jia, Energy-efficient joint task assignment and migration in data centers: A deep reinforcement learning approach, *IEEE Trans. Netw. Serv. Manag.* (2022).
- [39] P. Gupta, C.S. Rajpoot, An optimized energy-efficient task scheduling algorithm based on deep learning technique for energy consumption, *J. Chengdu Univ. Technol.* (2021).
- [40] C. He, Y. Yang, B. Hong, Cloud task scheduling based on policy gradient algorithm in heterogeneous cloud data center for energy consumption optimization, in: 2020 International Conference on Internet of Things and Intelligent Applications (ITIA), IEEE, 2020, pp. 1–5.
- [41] T. Oudaa, H. Gharsellaoui, S.B. Ahmed, An agent-based model for resource provisioning and task scheduling in cloud computing using DRL, *Procedia Comput. Sci.* 192 (2021) 3795–3804.
- [42] Z. Peng, J. Lin, D. Cui, Q. Li, J. He, A multi-objective trade-off framework for cloud resource scheduling based on the deep Q-network algorithm, *Cluster Comput.* 23 (2020) 2753–2767.
- [43] J. Uma, P. Vivekanandan, S. Shankar, Optimized intellectual resource scheduling using deep reinforcement Q-learning in cloud computing, *Trans. Emerg. Telecommun. Technol.* 33 (5) (2022) e4463.
- [44] Q. Zhang, M. Lin, L.T. Yang, Z. Chen, P. Li, Energy-efficient scheduling for real-time systems based on deep Q-learning model, *IEEE Trans. Sustain. Comput.* 4 (1) (2017) 132–141.
- [45] L. Zhang, K. Li, C. Li, K. Li, Bi-objective workflow scheduling of the energy consumption and reliability in heterogeneous computing systems, *Inform. Sci.* 379 (2017) 241–256.
- [46] Y. Qin, H. Wang, S. Yi, X. Li, L. Zhai, An energy-aware scheduling algorithm for budget-constrained scientific workflows based on multi-objective reinforcement learning, *J. Supercomput.* 76 (2020) 455–480.
- [47] K. Yu, Deep Reinforcement Learning Based DVFS Algorithm Frameworks (Ph.D. thesis), Swinburne University of Technology, 2022.
- [48] J. Li, X. Zhang, Z. Wei, J. Wei, Z. Ji, Energy-aware task scheduling optimization with deep reinforcement learning for large-scale heterogeneous systems, *CCF Trans. High Perform. Comput.* 3 (4) (2021) 383–392.
- [49] J. Zhao, M.A. Rodríguez, R. Buyya, A deep reinforcement learning approach to resource management in hybrid clouds harnessing renewable energy and task scheduling, in: 2021 IEEE 14th International Conference on Cloud Computing (CLOUD), IEEE, 2021, pp. 240–249.
- [50] T. SPEC, All published specpower_ss2008 results, 2023, URL http://www.spec.org/power_ss2008/results/power_ss2008.html.
- [51] R.S. Sutton, A.G. Barto, Reinforcement Learning: An Introduction, MIT Press, 2018.
- [52] M. Xu, A. N. Toosi, B. Bahrani, R. Razzaghi, M. Singh, Optimized renewable energy use in green cloud data centers, in: Service-Oriented Computing: 17th International Conference, ICSOC 2019, Toulouse, France, October 28–31, 2019, Proceedings 17, Springer, 2019, pp. 314–330.
- [53] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, M. Riedmiller, Playing atari with deep reinforcement learning, 2013, arXiv preprint arXiv:1312.5602.
- [54] T. Schaul, J. Quan, I. Antonoglou, D. Silver, Prioritized experience replay, 2015, arXiv preprint arXiv:1511.05952.
- [55] H. Van Hasselt, A. Guez, D. Silver, Deep reinforcement learning with double q-learning, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 30, 2016, pp. 2094–2100.
- [56] Z. Wang, T. Schaul, M. Hessel, H. van Hasselt, M. Lanctot, N. de Freitas, Dueling network architectures for deep reinforcement learning, 2015, arXiv preprint arXiv:1511.06581.
- [57] V. Mnih, A.P. Badia, M. Mirza, A. Graves, T. Harley, T.P. Lillicrap, D. Silver, K. Kavukcuoglu, Asynchronous methods for deep reinforcement learning, in: Proceedings of the 33rd International Conference on International Conference on Machine Learning-Volume 48, 2016, pp. 1928–1937.
- [58] J. Schulman, S. Levine, P. Abbeel, M. Jordan, P. Moritz, Trust region policy optimization, in: International Conference on Machine Learning, PMLR, 2015, pp. 1889–1897.
- [59] N. Heess, D. TB, S. Sriram, J. Lemmon, J. Merel, G. Wayne, Y. Tassa, T. Erez, Z. Wang, S. Eslami, et al., Emergence of locomotion behaviours in rich environments, 2017, arXiv preprint arXiv:1707.02286.
- [60] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, 2017, arXiv preprint arXiv:1707.06347.
- [61] T.P. Lillicrap, J.J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, D. Wierstra, Continuous control with deep reinforcement learning, 2015, arXiv preprint arXiv:1509.02971.
- [62] J. Yan, Y. Huang, A. Gupta, A. Gupta, C. Liu, J. Li, L. Cheng, Energy-aware systems for real-time job scheduling in cloud data centers: A deep reinforcement learning approach, *Comput. Electr. Eng.* 99 (2022) 107688.
- [63] Z. Chen, J. Hu, G. Min, C. Luo, T. El-Ghazawi, Adaptive and efficient resource allocation in cloud datacenters using actor-critic deep reinforcement learning, *IEEE Trans. Parallel Distrib. Syst.* 33 (8) (2021) 1911–1923.
- [64] A. Jayanetti, S. Halgamuge, R. Buyya, Deep reinforcement learning for energy and time optimized scheduling of precedence-constrained tasks in edge-cloud computing environments, *Future Gener. Comput. Syst.* 137 (2022) 14–30.
- [65] Y. Yang, C. He, B. Yin, Z. Wei, B. Hong, Cloud task scheduling based on proximal policy optimization algorithm for lowering energy consumption of data center, *KSII Trans. Internet Inf. Syst. (TIIS)* 16 (6) (2022) 1877–1891.
- [66] T. Thein, M.M. Myo, S. Parvin, A. Gawanmeh, Reinforcement learning based methodology for energy-efficient resource allocation in cloud data centers, *J. King Saud Univ.-Comput. Inf. Sci.* 32 (10) (2020) 1127–1139.
- [67] F. Li, B. Hu, Deepjs: Job scheduling based on deep reinforcement learning in cloud data center, in: Proceedings of the 4th International Conference on Big Data and Computing, 2019, pp. 48–53.
- [68] R.K. Devi, K.V. Devi, S. Arumugam, Dynamic batch mode cost-efficient independent task scheduling scheme in cloud computing, *Int. J. Adv. Soft Comput. Appl.* 8 (2) (2016).
- [69] N. Liu, Z. Li, J. Xu, Z. Xu, S. Lin, Q. Qiu, J. Tang, Y. Wang, A hierarchical framework of cloud resource allocation and power management using deep reinforcement learning, in: 2017 IEEE 37th International Conference on Distributed Computing Systems (ICDCS), IEEE, 2017, pp. 372–382.
- [70] M. Kumar, S.C. Sharma, PSO-COGENT: Cost and energy efficient scheduling in cloud environment with deadline constraint, *Sustain. Comput.: Inform. Syst.* 19 (2018) 147–164.
- [71] S. Mangalampalli, G.R. Karri, M. Kumar, Multi objective task scheduling algorithm in cloud computing using grey wolf optimization, *Cluster Comput.* (2022) 1–20.
- [72] M. Kumar, A. Kishor, J. Abawajy, P. Agarwal, A. Singh, A.Y. Zomaya, ARPS: An automatic resource provisioning and scheduling framework for cloud platforms, *IEEE Trans. Sustain. Comput.* 7 (2) (2021) 386–399.
- [73] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, et al., Pytorch: an imperative style, high-performance deep learning library, in: Proceedings of the 33rd International Conference on Neural Information Processing Systems, 2019, pp. 8026–8037.

- [74] M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G.S. Corrado, A. Davis, J. Dean, M. Devin, et al., Tensorflow: Large-scale machine learning on heterogeneous distributed systems, 2016, arXiv preprint arXiv:1603.04467.
- [75] A. Jain, A.A. Awan, Q. Anthony, H. Subramoni, D.K.D. Panda, Performance characterization of dnn training using tensorflow and pytorch on modern clusters, in: 2019 IEEE International Conference on Cluster Computing (CLUSTER), IEEE, 2019, pp. 1–11.
- [76] M.C. Chirodea, O.C. Novac, C.M. Novac, N. Bizon, M. Oproescu, C.E. Gordan, Comparison of tensorflow and pytorch in convolutional neural network-based applications, in: 2021 13th International Conference on Electronics, Computers and Artificial Intelligence (ECAI), IEEE, 2021, pp. 1–6.
- [77] R.N. Calheiros, R. Ranjan, A. Beloglazov, C.A. De Rose, R. Buyya, CloudSim: a toolkit for modeling and simulation of cloud computing environments and evaluation of resource provisioning algorithms, *Softw. - Pract. Exp.* 41 (1) (2011) 23–50.
- [78] L. Ruan, X. Xu, L. Xiao, F. Yuan, Y. Li, D. Dai, A comparative study of large-scale cluster workload traces via multiview analysis, in: 2019 IEEE 21st International Conference on High Performance Computing and Communications; IEEE 17th International Conference on Smart City; IEEE 5th International Conference on Data Science and Systems (HPCC/SmartCity/DSS), IEEE, 2019, pp. 397–404.
- [79] C. Reiss, J. Wilkes, J.L. Hellerstein, Google Cluster-Usage Traces: Format+ Schema, White Paper 1, Google Inc., 2011, pp. 1–14.
- [80] V. Sze, Y.-H. Chen, T.-J. Yang, J.S. Emer, Efficient processing of deep neural networks: A tutorial and survey, *Proc. IEEE* 105 (12) (2017) 2295–2329.
- [81] T.N. Sainath, B. Kingsbury, A.-r. Mohamed, G.E. Dahl, G. Saon, H. Soltau, T. Beran, A.Y. Aravkin, B. Ramabhadran, Improvements to deep convolutional neural networks for LVCSR, in: 2013 IEEE Workshop on Automatic Speech Recognition and Understanding, IEEE, 2013, pp. 315–320.
- [82] H. Sak, A.W. Senior, F. Beaufays, Long short-term memory recurrent neural network architectures for large scale acoustic modeling, Google Res. (2014).
- [83] H. Mao, M. Alizadeh, I. Menache, S. Kandula, Resource management with deep reinforcement learning, in: Proceedings of the 15th ACM Workshop on Hot Topics in Networks, 2016, pp. 50–56.
- [84] W. Chen, Y. Xu, X. Wu, Deep reinforcement learning for multi-resource multi-machine job scheduling, 2017, arXiv preprint arXiv:1711.07440.
- [85] B.-A. Han, J.-J. Yang, Research on adaptive job shop scheduling problems based on dueling double DQN, *IEEE Access* 8 (2020) 186474–186495.
- [86] S. Liang, Z. Yang, F. Jin, Y. Chen, Data centers job scheduling with deep reinforcement learning, in: Advances in Knowledge Discovery and Data Mining: 24th Pacific-Asia Conference, PAKDD 2020, Singapore, May 11–14, 2020, Proceedings, Part II 24, Springer, 2020, pp. 906–917.
- [87] N.K. Pandey, M. Diwakar, A. Shankar, P. Singh, M.R. Khosravi, V. Kumar, Energy efficiency strategy for big data in cloud environment using deep reinforcement learning, *Mob. Inf. Syst.* 2022 (2022).
- [88] G. Rjoub, J. Bentahar, O. Abdel Wahab, A. Saleh Bataineh, Deep and reinforcement learning for automated task scheduling in large-scale cloud computing systems, *Concurr. Comput.: Pract. Exper.* 33 (23) (2021) e5919.
- [89] J. Moolayil, Learn Keras for Deep Neural Networks-a Fast-Track Approach to Modern Deep Learning with Python, Springer, 2019.
- [90] S. Azizi, M. Shojafar, J. Abawajy, R. Buyya, Deadline-aware and energy-efficient IoT task scheduling in fog computing systems: A semi-greedy approach, *J. Netw. Comput. Appl.* 201 (2022) 103333.
- [91] Z. Tang, W. Jia, X. Zhou, W. Yang, Y. You, Representation and reinforcement learning for task scheduling in edge computing, *IEEE Trans. Big Data* 8 (3) (2020) 795–808.
- [92] S. Iftikhar, S.S. Gill, C. Song, M. Xu, M.S. Aslanpour, A.N. Toosi, J. Du, H. Wu, S. Ghosh, D. Chowdhury, et al., AI-based fog and edge computing: A systematic review, taxonomy and future directions, *Internet Things* (2022) 100674.
- [93] JamilBushra, IjazHumaira, ShojafarMohammad, MunirKashif, BuyyaRajkumar, Resource allocation and task scheduling in fog computing and internet of everything environments: A taxonomy, review, and future directions, *ACM Comput. Surv.* (2021).
- [94] Y. Zhang, R. Li, Y. Zhao, R. Li, Y. Wang, Z. Zhou, Multi-agent deep reinforcement learning for online request scheduling in edge cooperation networks, *Future Gener. Comput. Syst.* 141 (2023) 258–268.
- [95] H. Tran-Dang, S. Bhardwaj, T. Rahim, A. Musaddiq, D.-S. Kim, Reinforcement learning based resource management for fog computing environment: Literature review, challenges, and open issues, *J. Commun. Netw.* 24 (1) (2022) 83–98.
- [96] I. Ahmad, M.G. AlFailakawi, A. AlMutawa, L. Alsalmán, Container scheduling techniques: A survey and assessment, *J. King Saud Univ.-Comput. Inf. Sci.* (2021).
- [97] Z. Zhong, M. Xu, M.A. Rodriguez, C. Xu, R. Buyya, Machine learning-based orchestration of containers: A taxonomy and future directions, *ACM Comput. Surv.* 54 (10s) (2022) 1–35.
- [98] M.A. Rodriguez, R. Buyya, Container-based cluster orchestration systems: A taxonomy and future directions, *Softw. - Pract. Exp.* 49 (5) (2019) 698–719.
- [99] T. Menouer, KCSS: Kubernetes container scheduling strategy, *J. Supercomput.* 77 (5) (2021) 4267–4293.
- [100] B. Liu, J. Li, W. Lin, W. Bai, P. Li, Q. Gao, K-PSO: An improved PSO-based container scheduling algorithm for big data applications, *Int. J. Netw. Manag.* 31 (2) (2021) e2092.
- [101] E. Oró, V. Depoorter, A. Garcia, J. Salom, Energy efficiency and renewable energy integration in data centres. strategies and modelling review, *Renew. Sustain. Energy Rev.* 42 (2015) 429–445.
- [102] A. Agarwal, J. Sun, S. Noghbi, S. Iyengar, A. Badam, R. Chandra, S. Seshan, S. Kalyanaraman, Redesigning data centers for renewable energy, in: Proceedings of the Twentieth ACM Workshop on Hot Topics in Networks, 2021, pp. 45–52.
- [103] M. Xu, A.N. Toosi, R. Buyya, A self-adaptive approach for managing applications and harnessing renewable energy for sustainable cloud computing, *IEEE Trans. Sustain. Comput.* 6 (4) (2020) 544–558.
- [104] T. Yang, H. Tang, C. Bai, J. Liu, J. Hao, Z. Meng, P. Liu, Z. Wang, Exploration in deep reinforcement learning: a comprehensive survey, 2021, arXiv preprint arXiv:2109.06668.
- [105] T.T. Nguyen, V.J. Reddi, Deep reinforcement learning for cyber security, *IEEE Trans. Neural Netw. Learn. Syst.* (2021).
- [106] C. Dwork, Differential privacy: A survey of results, in: International Conference on Theory and Applications of Models of Computation, Springer, 2008, pp. 1–19.
- [107] Q. Miao, H. Lin, X. Wang, M.M. Hassan, Federated deep reinforcement learning based secure data sharing for internet of things, *Comput. Netw.* 197 (2021) 108327.
- [108] S.S. Gill, M. Xu, C. Ottaviani, P. Patros, R. Bahsoon, A. Shaghagh, M. Golec, V. Stankovski, H. Wu, A. Abraham, et al., AI for next generation computing: Emerging trends and future directions, *Internet Things* 19 (2022) 100514.



Huanhuan Hou received her B.S. degree in Network Engineering from the North University of China. She is currently studying at Universiti Teknologi MARA (UiTM) for Ph.D. in Computer Science. Her research interests include task scheduling, cloud computing, and reinforcement learning.



Siti Nuraishah Agos Jawaddi received a B.Sc. degree in Computer Science from Universiti Teknologi MARA (UiTM), Malaysia. She is currently enrolled in M.Sc in Computer Science. Her research interests include Microservices, Cloud Computing, and Software Verification.



Azlan Ismail completed his B.Sc. and M.Sc. degrees in computer science at Universiti Teknologi Malaysia. Following that, he obtained his Ph.D. from the University of Wollongong in Australia in 2012. Presently, he holds the position of a senior fellow at the Institute for Big Data Analytics and Artificial Intelligence (IBDAI) at Universiti Teknologi MARA (UiTM). Additionally, he serves as an Associate Professor in the School of Computing Sciences, College of Computing, Informatics and Mathematics, also at UiTM. During his postdoctoral study at the Department of Computer Science at the University of Oxford, he was sponsored by the Ministry of Higher Education in Malaysia. His research interests encompass Cloud Computing, Machine Learning, Self-adaptive Systems, and Software Verification. He has actively contributed as a reviewer for international journals and conference proceedings.